

6

ARTIFICIAL NEURAL NETWORKS: BASIC CONCEPTS

Key Concepts

Neuron, dendrite, soma, axon, synapse, artificial neuron, artificial neural network (ANN), processing element (PE), activation function, activation, pattern, pattern classification, perceptron, pattern association, Hopfield network, parallel relaxation, McCulloch–Pitts neural model, linear separability, decision plane, decision surface, perceptron convergence theorem, XOR problem, single layer feed forward, multi-layer feed forward, hidden layer, competitive ANN, recurrent network, identity function, step function, threshold function, heaviside function, sigmoid function, binary sigmoid, logistic sigmoid, steepness parameter, hyperbolic tangent function, supervised learning, unsupervised learning, training vector, perceptron learning, delta learning, least-mean square (LMS) learning, correlation learning, outstar learning, Hebb rule, Heb learning, epoch of learning, learning rate, Widrow–Hoff rule, extended delta rule, winner takes all, exemplar, code-book vector, Euclidean distance

Chapter Outline

- | | |
|--------------------------------------|---------------------------------|
| 6.1 Introduction | 6.7 Learning by ANNs |
| 6.2 Computation in terms of patterns | Chapter summary |
| 6.3 The McCulloch–Pitts neural model | Bibliography & historical notes |
| 6.4 The perceptron | Solved problems |
| 6.5 ANN architectures | Test your knowledge |
| 6.6 Activation functions | Exercise |

Artificial neural networks (ANNs) follow a computational paradigm that is inspired by the structure and functionality of the brain. The brain is composed of nearly 100 billion neurons each of which is locally connected to its neighbouring neurons. The biological neuron possesses very elementary signal processing capabilities like summing up the incoming signals and then propagating to its neighbours depend-

ing on certain conditions. However the sum total of the parallel and concurrent activities of these 100 billion neurons gives rise to the highly sophisticated, complex, and mysterious phenomenon that we call 'consciousness'. This chapter provides an overview of the basic concepts of ANNs. Unlike mainstream computation where information is stored as localized bits, an ANN preserves information as weights of interconnections among its processing units. Thus, as in the brain, information in ANNs too resides in a distributed manner, resulting in greater fault tolerance. Moreover, multiple information may be superimposed on the same ANN for storage purpose. We will see that like the human brain, ANNs also perform computation in terms of patterns rather than data. This chapter presents the major artificial neural models, as well as the various ANN architectures and activation functions. The basic learning strategies are also discussed in this chapter.

6.1 INTRODUCTION

Computation is generally perceived as a sequence of operations that transforms a set of input data to a desired result. The agent that carries out the computation is either an intelligent human being or a typical Von Neumann computer consisting of a processor and a memory, along with the input and the output units (Fig. 6.1). The memory contains both instruction and data. Computation is accomplished by the CPU through a sequence of fetch and execute cycles.

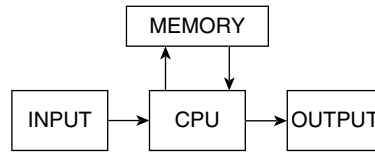


Fig. 6.1. Block diagram of a stored program computer

The sequential fetch and execute model of computation, based on a single-processor stored programme digital computer as the hardware platform, has been immensely successful in the history of computers. Its notorious member-crunching and symbol-manipulating capacity has rendered it an indispensable tool for the civilized world. Innumerable applications in everyday activities and other enterprises, *e.g.* commerce, industry, communication, management, governance, research, entertainment, healthcare, and so on, were developed, are being developed, and shall be developed on the basis of the this model.

However, the power of such a computational model is not unchallenged. There are activities that a normal human being may require a fraction of a second to perform while it would take ages by even the fastest computer. Take, for example, the simple task of recognizing the face of a known person. We do it effortlessly, in spite of the infinite variations due to distance, angle of vision, lighting, posture, distortion due to mood, or emotion of the person concerned, and so on. Occasionally, we recognize a face after a gap of, say, twenty years, even though the face has changed a lot through aging, and has little semblance to its appearance of twenty year ago. This is still very difficult, if not impossible, to achieve for a present day computer.

Where does the power of a human brain, vis-à-vis a stored programme digital computer, lie? Perhaps it lies in the fact that we, the human beings, do not *think* in terms of *data* as a computer does, but in terms of *patterns*. When we look at a face, we never think in terms of the pixel values, but perceive the face as a whole, as a pattern. Moreover, the structure of the human brain is drastically different from

the Von-Neuman architecture. Instead of one, or a few, processors, the brain consists of 100 billion interconnected cells called the neurons. Individually, a neuron can do no more than some primitive tasks like collecting stimuli from the neighboring neurons and then passing them on to other neighbouring neurons after some elementary processing. But the *sum* of these simultaneous activities of 100 billion neurons is what we call the human consciousness. Artificial neural network, occasionally abbreviated as ANN, is an alternative model of computation that is inspired by the structure and functionality of the human brain.

6.1.1 The Biological Neuron

The building block of a human brain is the biological neuron. The main parts of the cross-section of a common biological neuron and their functions are shown in Fig. 6.2.

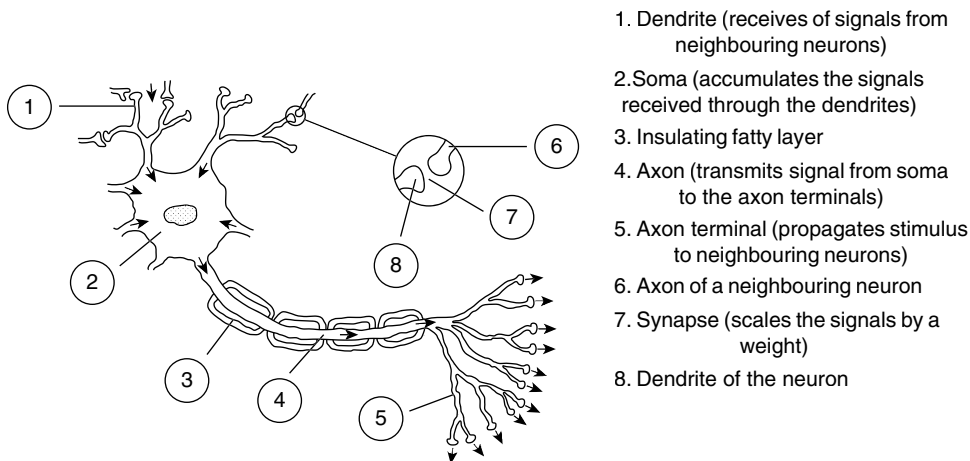


Fig. 6.2. Structure of a biological neuron

It consists of three primary parts, *viz.*, the dendrites, soma, and the axon. The dendrites collect stimuli from the neighbouring neurons and pass it on to soma which is the main body of the cell. The soma accumulates the stimuli received through the dendrites. It 'fires' when sufficient stimuli is obtained. When a neuron fires it transmits its own stimulus through the axon. Eventually, this stimulus passes on to the neighboring neurons through the axon terminals. There is a small gap between the end of an axon terminal and the adjacent dendrite of the neighbouring neuron. This gap is called the synapse. A nervous stimulus is an electric impulse. It is transmitted across a synaptic gap by means of electrochemical process.

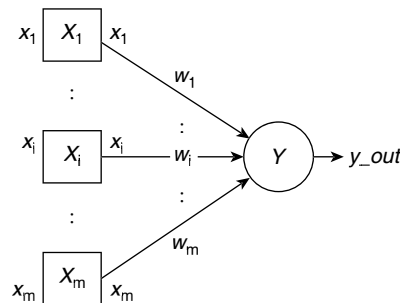
The synaptic gap has an important role to play in the activities of the nervous system. It scales the input signal by a weight. If the input signal is x , and the synaptic weight is w , then the stimulus that finally reaches the soma due to input x is the product $x \times w$. The significance of the weight w provided by the synaptic gap lies in the fact that this weight, together with other synaptic weights, embody the knowledge stored in the network of neurons. This is in contrast with digital computers where the knowledge is stored as a programme in the memory. The salient features of biological neurons are summarized in Table 6.1.

Table 6.1. *Salient features of a biological neuron*

#	Feature
1	The body of the neuron is called the <i>soma</i> that acts as a processing element to receive numerous signals through the dendrites simultaneously.
2	The strengths of the incoming signals are modified by the synaptic gaps.
3	The role of the soma, i.e., the processing element of a neuron, is simple. It sums up the weighted input signals and if the sum is sufficiently high, it transmits an output signal through the axon. The output signal reaches the receiving neurons in the neighbourhood through the axon terminals.
4	The weight factors provided by the synaptic gaps are modified over time and experience. This phenomenon, perhaps, accounts for development of skills through practice, or loss of memory due to infrequent recall of stored information.

6.1.2 The Artificial Neuron

An artificial neuron is a computational model based on the structure and functionality of a biological neuron. It consists of a processing element, a number of inputs and weighted edges connecting each input to the processing element (Fig. 6.3). A processing unit is usually represented by a circle, as indicated by the unit Y in Fig. 6.3. However, the input units are shown with boxes to distinguish them from the processing units of the neuron. This convention is followed throughout this book. An artificial neuron may consist of m number of input units $X_1, X_2, \dots, X_m$. In Fig. 6.3 the corresponding input signals are shown as $x_1, x_2, \dots, x_m$, and y_out is the output signal of the processing unit Y .

**Fig. 6.3** Structure of an artificial neuron

The notations used in Fig. 6.3 are summarized in Table 6.2. These notational conventions are followed throughout this text unless otherwise stated.

Table 6.2. *Notational convention*

Symbol Used	Description
X_i	The i th input unit.
Y	The output unit. In case there are more than one output units, the j th output unit is denoted as Y_j .
x_i	Signal to the input unit X_i . This signal is transmitted to the output unit Y scaled by the weight w_i .

(Continued)

Table. 6.2. *Continued*

Symbol Used	Description
w_i	The weight associated with the interconnection between input unit X_i and the output unit Y . In case there are more than one output units, w_{ij} denotes the weight between input unit X_i and the j th output unit Y_j .
y_in	The total (or net) input to the output unit Y . It is the algebraic sum of all weighted inputs to Y .
y_out	Signal transmitted by the output unit Y . It is known as the activation of Y .

The net input y_in to the processing element Y is obtained as

$$y_in = x_1w_1 + x_2w_2 + \cdots + x_mw_m = \sum_{i=1}^m x_i w_i \quad (6.1)$$

If there are more than one output units, then the net input to the j th output unit Y_j , denoted as y_in_j , is given by

$$y_in_j = x_1w_{1j} + x_2w_{2j} + \cdots + x_mw_{mj} = \sum_{i=1}^m x_i w_{ij} \quad (6.2)$$

The weight w_i associated with the input X_i may be positive, or negative. A positive weight w_i means the corresponding input X_i has an excitatory effect on Y . If, however, w_i is negative then the input X_i is said to have an inhibitory effect on Y . The output signal transmitted by Y is a function of the net input y_in . Hence,

$$y_out = f(y_in) \quad (6.3)$$

In its simplest form $f(.)$ is a *step function*. A *binary step function* for the output unit is defined as

$$y_out = f(y_in) = \begin{cases} 1, & \text{if } y_in > 0, \\ 0, & \text{if } y_in \leq 0. \end{cases} \quad (6.4)$$

Taking Equation 6.1 into consideration we get

$$y_out = f(y_in) = \begin{cases} 1, & \text{if } \sum_{i=1}^m x_i w_i > 0, \\ 0, & \text{if } \sum_{i=1}^m x_i w_i \leq 0. \end{cases} \quad (6.5)$$

When a non-zero threshold θ is used Equations 6.4 and 6.5 take the forms

$$y_out = f(y_in) = \begin{cases} 1, & \text{if } y_in > \theta, \\ 0, & \text{if } y_in \leq \theta. \end{cases} \quad (6.6)$$

Or,

$$y_out = f(y_in) = \begin{cases} 1, & \text{if } \sum_{i=1}^m x_i w_i > \theta, \\ 0, & \text{if } \sum_{i=1}^m x_i w_i \leq \theta. \end{cases} \quad (6.7)$$

The function $f(\cdot)$ is known as the activation function of the neuron, and the output y_{out} is referred to as the activation of Y . Various activation functions are discussed in later parts of this chapter.

The structure of an artificial neuron is simple, like its biological counterpart. Its processing power is also very limited. However, a network of artificial neurons, popularly known as ANN has wonderful capacities. The computational power of ANNs is explored in the subsequent chapters with greater details.

6.1.3 Characteristics of the Brain

Since brain is the source of inspiration, as well as the model that ANNs like to follow and achieve, it is worthwhile to ponder over the characteristics of a brain as a computing agent. The most striking feature of a brain is its extremely parallel and decentralized architecture. It consists of more or less 100 billion neurons interconnected among them. The interconnections are local in the sense that each neuron is connected to its neighbours but not to a neuron far away. There is practically no centralized control in a brain. The neurons act on the basis of local information. These neurons function in parallel mode and concurrently. Apparently the brain is very slow compared to the present day computers. This is due to the fact that neurons operate at milliseconds range while the modern VLSI microchip process signals at nanosecond scale of time. The power of the brain lies not in the signal processing speed of its neuron but in the parallel and concurrent activities of 100 billion neurons. Another fascinating fact about the brain is its fault tolerance capability. As the knowledge is stored inside the brain in a distributed manner it can restore knowledge even when a portion of the brain is damaged. A summary of the essential features of a brain is presented in Table 6.3.

Table 6.3. *Essential features of a brain*

#	Aspect	Description
1	Architecture	The average human brain consists of about 100 billion neurons. There are nearly 10^{15} number of interconnections among these neurons. Hence the brain's architecture is highly connected.
2	Mode of operation	The brain operates in extreme parallel mode. This is in sharp contrast with the present day computers which are essentially single-processor machines. The power of the brain lies in the simultaneous activities of 100 billion neurons and their interactions.
3	Speed	Very slow, and also very fast. Very slow in the sense that neurons operate at milliseconds range which is miserably slow compared to the speed of present day VLSI chips that operate at nanoseconds range. So computers are tremendously fast and flawless in number crunching and data processing tasks compared to human beings. Still the brain can perform activities in split-seconds (e.g., converse in natural language, carry out common sense reasoning, interpret a visual scenery, etc.) which a modern supercomputer may take ages to carryout.
4	Fault tolerance	The brain is highly fault tolerant. Knowledge is stored within the brain in a distributed manner. Consequently, if a portion of the brain is damaged, it can still go on functioning by retrieving or regenerating the lost knowledge from the remaining neurons.

(Continued)

Table 6.3. Continued

#	Aspect	Description
5	Storage mechanism	The brain stores information as strengths of the interconnections among the neurons. This accounts for the adaptability of the brain. New information can be added by adjusting the weights without destroying the already stored information.
6	Control	There is no global control in the brain. A neuron acts on local information available with its neighbouring neurons. The neurons pass on the results of processing only to the neurons adjacent to them.

6.2 COMPUTATION IN TERMS OF PATTERNS

It was observed earlier in this chapter that the brain perceives the physical world in terms of *patterns*, rather than *data*. Since ANNs are inspired by the brain, both structurally and behaviourally, it is worthwhile to consider the nature of pattern-oriented computation *vis-a-vis* computation on the basis of stored programme. Two fundamental operations relating to patterns, pattern classification and pattern association, are explained in this subsection with the help of simple illustrative examples.

6.2.1 Pattern Classification

Classification is the process of identifying the class to which a given pattern belongs. For example, let us consider the set S of all 3-bit patterns. We may divide the patterns of S into two classes A and B where A is the class of all patterns having more 0s than 1s and B the converse. Therefore

$$S = \{000, 001, 010, 011, 100, 101, 110, 111\}$$

$$A = \{000, 001, 010, 100\}$$

$$B = \{011, 101, 110, 111\}$$

Now, given an arbitrary 3-bit pattern, the classification problem here is to decide whether it belongs to the class A , or class B . In other words, we have to establish the mapping shown in Fig. 6.4.

The simplest way to achieve this is to execute a table look-up procedure, as shown in Fig. 6.5(a). All we have to do is to find the appropriate row in the table corresponding to the given pattern and read the class name to which it belongs. However, creation and storage of the table becomes impractical as the volume of stored patterns increases. In practice, we may have to deal with billions of multidimensional patterns.

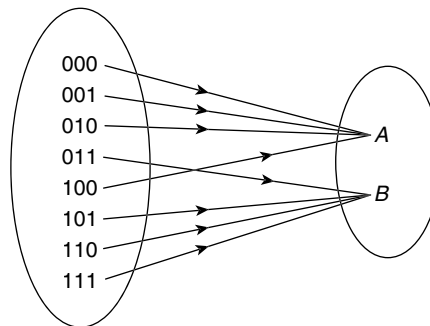


Fig. 6.4. Classification of 3-bit patterns based on the number of 0s and 1s.

Fig. 6.5(b) presents a conventional computer program to perform this task. The program is written in a pseudo-language. The technique is to count the number of 0s and 1s in the given patterns and store them in the local variables n_0 and n_1 . Then depending on whether $n_0 > n_1$ or $n_0 < n_1$ the programme returns the class name A or B respectively. The algorithm has a time complexity of $O(n_b)$ where n_b is the number of bits in the given pattern.

Row #	Pattern			Class
0	0	0	0	A
1	0	0	1	A
2	0	1	0	A
3	0	1	1	B
4	1	0	0	A
5	1	0	1	B
6	1	1	0	B
7	1	1	1	B

(a) Classification as a table look-up procedure

Procedure Classify (x, A, B)

Begin

$n_0 = n_1 = 0$; /* initialize counts */

/* count 0s and 1s in x */

For $i \leftarrow 1$ **to** 3 **do**

If the i^{th} bit is 0 **Then**

n_0^{++} ; **Else** n_1^{++} ;

End-if

End-for

If $n_0 > n_1$ **Then** Return A;

Else Return B;

End-if

End-procedure

(b) A procedure for classification of 3-bit patterns

Fig. 6.5. Two methods for classification of 3-bit patterns

Is it possible to solve the problem in a different way? Fig. 6.6 shows an artificial neuron with three inputs x_1, x_2, x_3 connected to a processing element Y through the weight factors as shown in the figure. It is a simplified version of a neural model called perception, proposed by Rosenblatt in 1962.

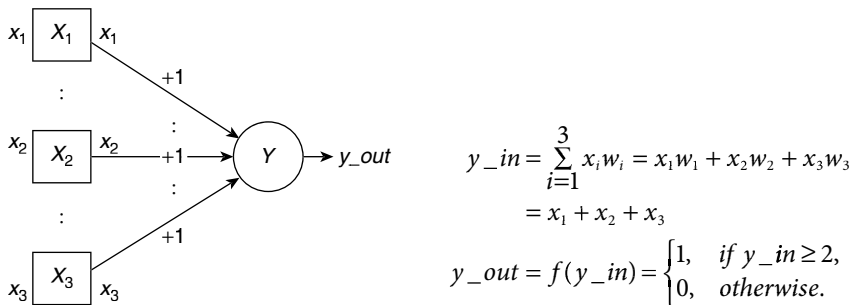


Fig. 6.6 An artificial neuron to classify 3-bit binary patterns based on the number of 0s and 1s

The weights w_1, w_2, w_3 associated with the interconnection paths from the inputs x_1, x_2, x_3 to Y are chosen in a way that the net input y_{in} to Y is greater than or equal to 2 when there are two or more 1s among x_1, x_2, x_3 . The activation y_{out} then becomes 1, indicating that the patterns belongs to the class

B. On the other hand, when there are more 0s than 1s, the net (total) input $y_{in} = \sum_{i=1}^3 w_i x_i \leq 1 < 2$, so

that the output signal, the *activation*, is $y_{out} = 0$. This implies that the given pattern belongs to the class A. Time complexity of this method is $O(1)$ because the inputs are fed to the processing element parallelly.

An alternative arrangement of two output nodes Y_1 and Y_2 to solve the same classification problem is shown in Fig. 6.7. Here both the units Y_1 and Y_2 are connected to the same inputs x_1, x_2, x_3 and these units explicitly correspond to the classes A and B respectively. If the input pattern belongs to the class A then Y_1 fires (i.e., computes the activation as $y_{out_1} = 1$), otherwise, unit Y_2 fires. Y_1 and Y_2 never fire simultaneously.

It should be noted that Fig. 6.5(a) and 6.5(b) presents the classification knowledge in the form of an algorithm. On the other hand, the ANN counterpart of the concerned classification task embodies the same knowledge in the form of certain interconnection weights and the activation functions. If we change the weights, or the activation functions $f(y_{in})$, the capability of the ANN changes accordingly.

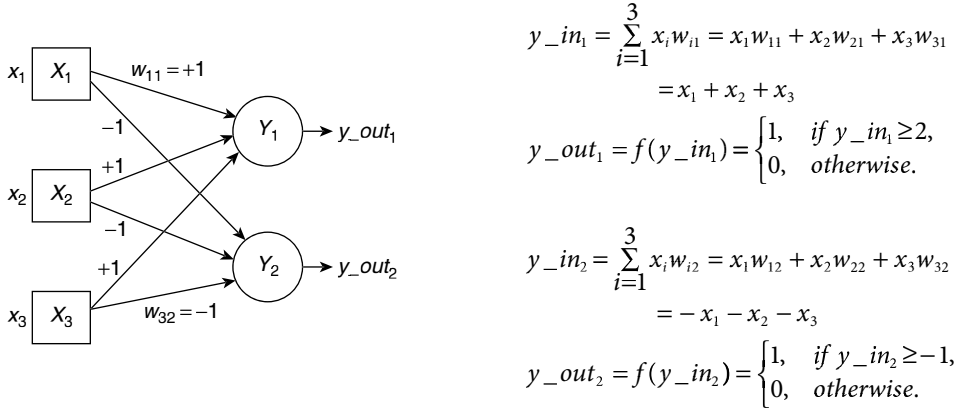


Fig. 6.7. Classification of 3-bit patterns with two output units

6.2.2 Pattern Association

Recognizing an input pattern as one among several patterns already stored in memory, in case such a match exists, is an act that we, the human beings, carry out effortlessly in our daily life. Examples of such acts are recognition of a known face, handwritten characters, verbal speeches, the fragrance of a flower etc. All of these are instances of pattern association. A person, with his eyes closed, can recognize a rose by its fragrance. How does it happen? It seems that various odours are already stored in our memory. When we smell an odour, our brain tries to match this sensation with its stored odours. It returns the nearest match and this correspond to recognizing the odour, or the sensation in general. However, in case there is no match between the input sensation and any of the stored patterns – we fail to recognize the input, or, to be more precise, we conclude that the input is unknown to us.

Given an input pattern, and a set of patterns already stored in the memory, finding the closest match of the input pattern among the stored patterns and returning it as the output, is known as pattern association. The basic concept of pattern association is explained below with the help of a simple illustrative example. The example is inspired by Hopfield network [1982] which is discussed later in greater details.

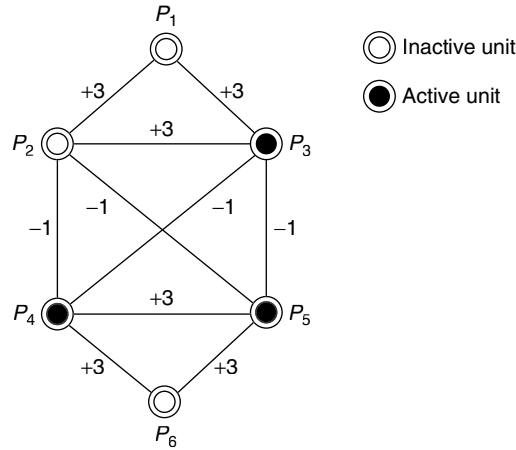


Fig. 6.8. A network for pattern recognition

Let us consider a network of six processing elements (PEs) or units as shown in Fig. 6.8. The essential features of the network are described below.

- i) *PE states* At any instant, a unit may either be in an *active* or an *inactive* state. Moreover, depending on the circumstances, the state of a unit may change from active to inactive, and vice versa. In Fig. 6.8 an active unit is shown with a black circle and an inactive unit is indicated by a hollow circle.
- ii) *Interconnections* All interconnections are bidirectional. Magnitude of the weight associated with an interconnection gives the strength of influence the connected units play on each other.
- iii) *Signed weights* A negative weight implies that the corresponding units tend to inhibit, or deactivate, each other. Similarly, positively interconnected units tend to activate each other.
- iv) *Initialization* The network is initialized by making certain units active and keeping others inactive. The initial combination of active and inactive units is considered as the input pattern. After initialization, the network passes through a number of transformations. The transformations take place according to the rules described below.
- v) *Transformations* At each stage during the sequence of transformations, the next state of every unit P_i , $i = 1, \dots, 6$, is determined. The next state of a unit P_i is obtained by considering all *active* neighbours of P_i and taking the algebraic sum of the weights of the paths between P_i and the neighbouring active units. If the sum is greater than 0, then P_i becomes active for the next phase. Otherwise it becomes inactive. The state of a unit without any active unit in its neighbourhood remains unaltered. This process is known as parallel relaxation.

For example, let the network be initialized with the pattern shown in Fig. 6.9(a). Initially, all units except P_2 and P_5 are inactive. To find the state of P_1 in the next instant, we look for the active neighbours of P_1 and find that P_2 is the only active unit connected to P_1 through an interconnection link of weight +3. Hence P_1 becomes active in the next instant. Similarly, for P_3 , both P_2 and P_5 are active units in its neighbourhood. The sum of the corresponding weights is $w_{23} + w_{35} = +3 - 1 = +2$. Hence P_3 also becomes

active. However P_2 itself becomes inactive because the only active unit in its vicinity, P_5 , is connected to it through a negatively weighted link. Table 6.4 shows the details of computations for transformation of the network from Fig. 6.9(a) to Fig. 6.9(b). The configuration of Fig. 6.9(b) is not stable. The network further transforms itself from Fig. 6.9(b) to Fig. 6.9(c), which is a stable state. Therefore, we can say that the given network associates the pattern shown in Fig. 6.9(a) to that shown in Fig. 6.9(c).

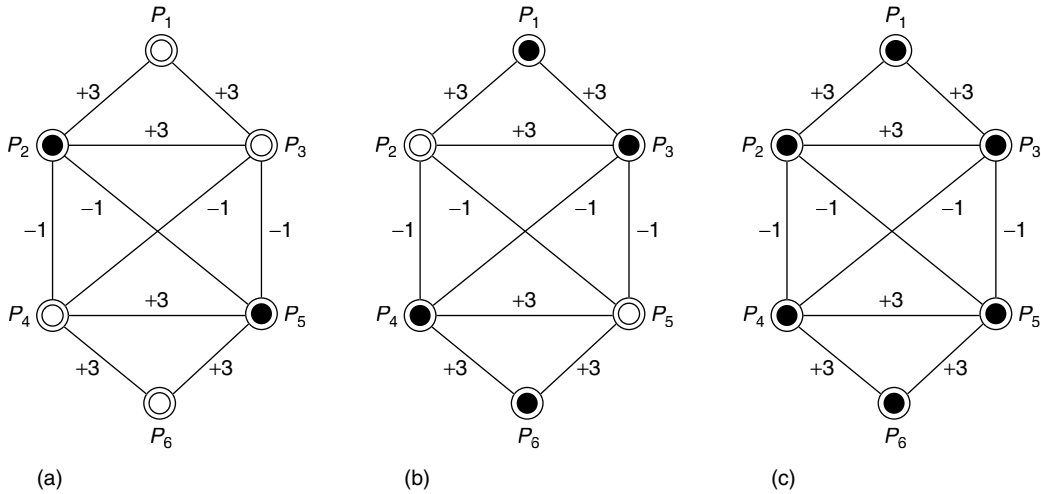


Fig. 6.9. Pattern association through parallel relaxation

A little investigation reveals that the given network has three non-trivial stable states as shown in Fig. 6.10(a)-(c). The trivial state is that where all units are inactive. It can be easily verified that if one or more of the units P_1, P_2, P_3 is/are active initially while the rest, P_4, P_5, P_6 are inactive, the network converges to the pattern shown in Fig. 6.10(a). Similarly, the pattern of Fig. 6.10(b) is associated with any input pattern where at least one unit of the group $\{P_4, P_5, P_6\}$ is/are active. Finally, an input pattern having active units from both the groups $\{P_1, P_2, P_3\}$ and $\{P_4, P_5, P_6\}$ would associate to with the patterned depicted in Fig. 6.10(c). Hence, the given network may be thought of as storing three non-trivial patterns as discussed above. Such networks are also referred to as associative memories, or content-addressable memories.

Table 6.4. Computation of parallel relaxation on fig. 6.9 (a)

Unit	Present State	Active neighbouring unit(s)	Sum	Next State
P_1	Inactive	P_2	+3	Active
P_2	Active	P_5	-1	Inactive
P_3	Inactive	P_2, P_5	+3 - 1 = +2	Active
P_4	Inactive	P_2, P_5	-1 + 3 = +2	Active
P_5	Active	P_2	-1	Inactive
P_6	Inactive	P_5	+3	Active

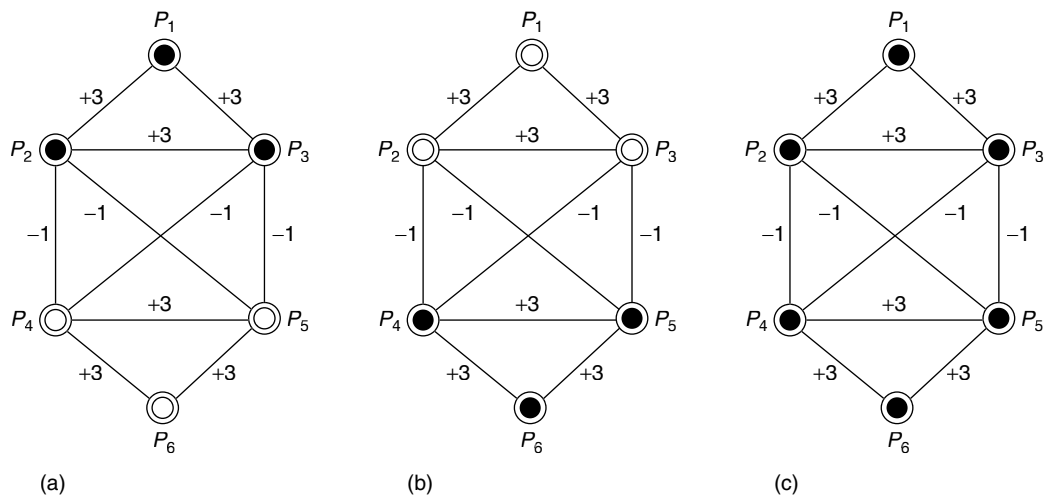


Fig. 6.10. Non-trivial patterns stored in a Hopfield network

6.3. THE MCCULLOCH-PITTS NEURAL MODEL

The earliest artificial neural model was proposed by McCulloch & Pitts in 1943. Fig. 6.11 depicts its structure. It consists of a number of input units connected to a single output unit. The interconnecting links are unidirectional. There are two kinds of inputs, namely, excitatory inputs and inhibitory inputs. The salient features of a McCulloch & Pitts neural net are summarized in Table 6.5.

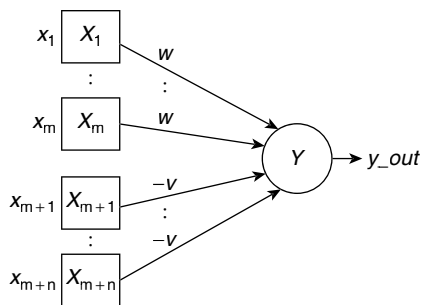


Fig. 6.11. Structure of a McCulloch-Pitts Neuron

Table 6.5. Salient features of mcculloch-pitts artificial neuron

- 1 There are two kinds of input units, *excitatory*, and *inhibitory*. In Fig. 6.11 the excitatory inputs are shown as inputs $X_1, \dots, X_m$ and the inhibitory inputs are $X_{m+1}, \dots, X_{m+n}$. The excitatory inputs are connected to the output unit through positively weighted links. Inhibitory inputs have negative weights on their connecting paths to the output unit.
- 2 All excitatory weights have the same positive magnitude w and all inhibitory weights have the same negative magnitude $-v$.

(Continued)

Table 6.5. *Continued*

3	The activation $y_{out} = f(y_{in})$ is binary, i.e., either 1 (in case the neuron fires), or 0 (in case the neuron do not fire).
4	The activation function is a binary step function. It is 1 if the net input y_{in} is greater than or equal to a given threshold value θ , and 0 otherwise.
5	The inhibition is absolute. A single inhibitory input should prevent the neuron from firing irrespective of the number of active excitatory inputs.

The net input y_{in} to the neuron Y is given by

$$y_{in} = \sum_{i=1}^m x_i \times w + \sum_{j=m+1}^n x_j \times (-v) = w \sum_{i=1}^m x_i - v \sum_{j=m+1}^n x_j \quad (6.8)$$

If θ be the threshold value, then the activation of Y , i.e., y_{out} , is obtained as

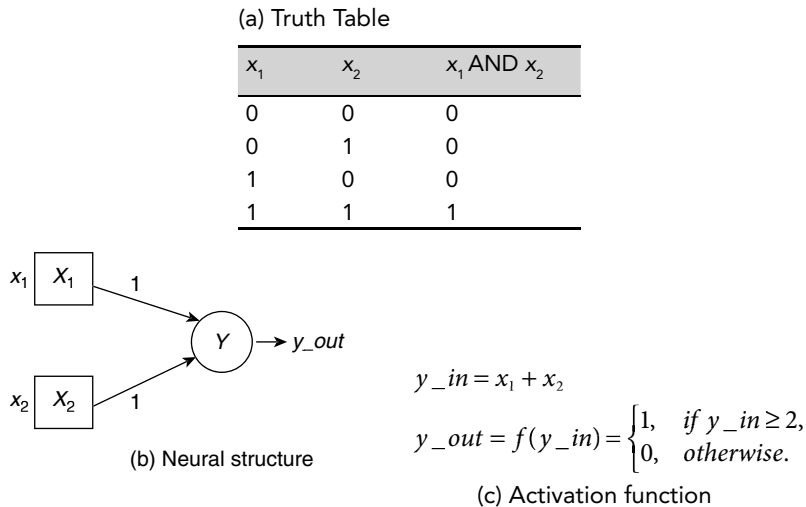
$$y_{out} = f(y_{in}) = \begin{cases} 1, & \text{if } y_{in} \geq \theta, \\ 0, & \text{otherwise.} \end{cases} \quad (6.9)$$

To ensure absolute inhibition, y_{in} should be less than the threshold even when a single inhibitory input is on while none of the excitatory inputs are off. Assuming that the inputs are binary, i.e., 0 or 1, the criterion of absolute inhibition requires

$$y_{in} = w \sum_{i=1}^m x_i - v < \theta \quad (6.10)$$

$$\therefore w \times m - v < \theta. \quad (6.11)$$

Simple McCulloch-Pitts neurons can be designed to perform conventional logical operations. For this purpose one has to select the appropriate number of inputs, the inter connection weights and the appropriate activation function. A number of such logic-performing McCulloch-Pitts neural nets are presented below as illustrative examples.


Fig. 6.12. A McCulloch-Pitts neuron to implement logical AND operation

Example 6.1 (Implementation of logical AND with McCulloch-Pitts neural model)

Fig. 6.12 shows a McCulloch-Pitts neuron to perform the logical AND operation. It should be noted that all inputs in Fig. 6.12(b) are excitatory. No inhibitory input is required to implement the logical AND operation. The interconnection weights and the activation functions are so chosen that the output is 1 if and only if both the inputs are 1, otherwise it is 0.

Example 6.2 (Implementation of logical OR with McCulloch-Pitts neural model)

The McCulloch-Pitts neuron to perform logical OR operation is shown in Fig. 6.13. It is obvious from the figure that the neuron outputs a 1 whenever there is at least one 1 at its inputs. The neuron is structurally identical to the AND-performing neuron. Only the activation function is changed appropriately so that the desired functionality is ensured.

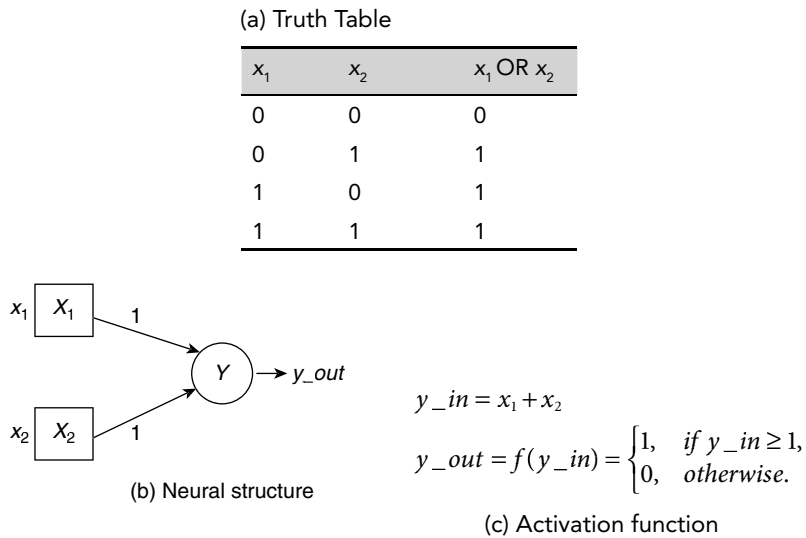


Fig. 6.13. A McCulloch-Pitts Neuron to implement logical OR operation

Example 6.3 (Implementation of logical AND-NOT with McCulloch-Pitts neural model)

The logical AND-NOT operation is symbolically expressed as $x_1 \cdot x_2'$, or $x_1 \text{ AND } (\text{NOT } x_2)$. It produces a 1 at the output only when x_1 is 1 and x_2 is 0. The McCulloch-Pitts neuron to perform this operation is shown in Fig. 6.14. The inhibitory effect of x_2 is implemented by attaching a negative weight to the path between x_2 and Y. The arrangement ensures that the output is 1 only when $x_1 = 1$ and $x_2 = 0$. For all other combinations of x_1 and x_2 the output is 0.

Example 6.4 (Implementation of logical XOR with McCulloch-Pitts neural model)

As in digital logic design, simple McCulloch-Pitts neurons performing basic logical operations can be combined together to implement complex logic functions. As an example, Fig. 6.15 shows

the implementation of the *XOR* function with two *AND-NOT* operations. Unlike the previous examples, here we had to implement the function with the help of a network of neurons, rather than a single neuron. Moreover, the neurons are placed at various levels so that the outputs from a lower level are fed as inputs to the next level of neurons. All the three processing elements, Y_1 , Y_2 , and Z , have the same activation function as described in Fig. 6.15 (c). The *net_in* is the net input to a processing element. For example, *net_in* for Y_1 , is $y_{in_1} = x_1 - x_2$. The same for the units Y_2 and Z are $y_{in_2} = -x_1 + x_2$, and $z_{in} = y_{out_1} + y_{out_2}$ respectively, where y_{out_1} and y_{out_2} are the respective activations of the units Y_1 and Y_2 .

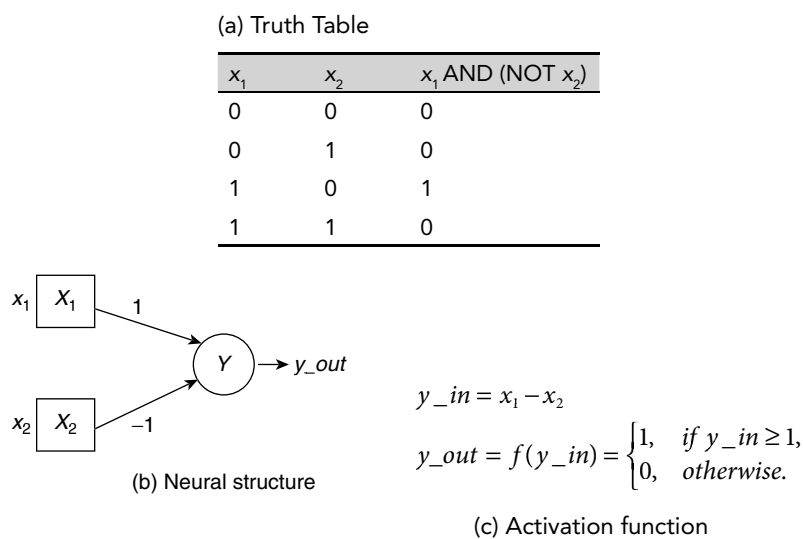


Fig. 6.14. Logical *AND-NOT* operation with a McCulloch-Pitts neuron

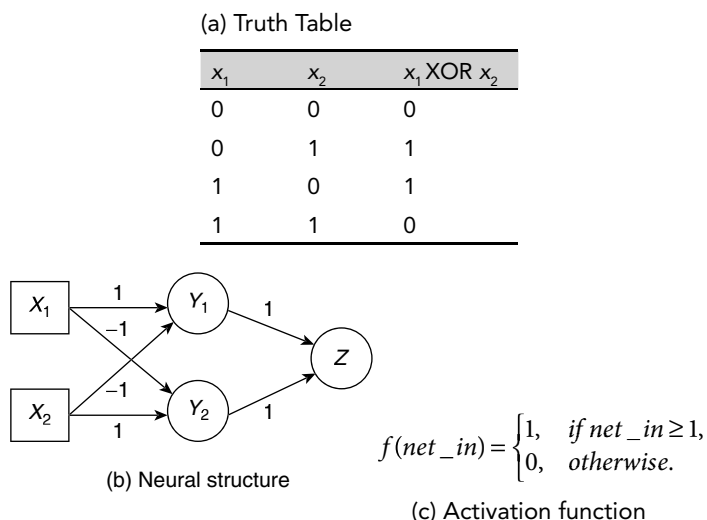


Fig. 6.15. McCulloch-Pitts neural network to implement logical *XOR* operation

Example 6.5 Consider the McCulloch-Pitts neural network shown in Fig. 6.16. All the units, except those at the input level, have the activation function

$$f(x) = \begin{cases} 1, & \text{if } x \geq 2, \\ 0, & \text{otherwise.} \end{cases}$$

What are the responses of the output unit Z with respect to various input combinations. We assume the inputs are binary. What logical function the whole network realizes?

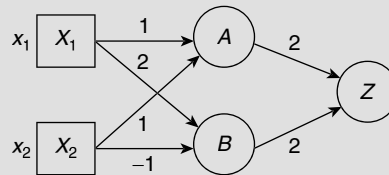


Fig. 6.16. A McCulloch-Pitts neural network

Let us first compute the responses of the intermediate units A and B . The responses of the output unit Z will be determined subsequently.

a) Responses of unit A

Inputs		Net input to A ($A_{in} = x_1 + x_2$)	Activation of A (A_{out})	Logic function realized
x_1	x_2			
0	0	0	0	
0	1	1	0	
1	0	1	0	
1	1	2	1	

b) Responses of unit B

Inputs		Net input to B ($B_{in} = 2x_1 - x_2$)	Activation of B (B_{out})	Logic function realized
x_1	x_2			
0	0	0	0	
0	1	-1	0	
1	0	2	1	
1	1	1	0	

c) Responses of unit Z

Inputs		A_out	B_out	Net input to Z ($Z_{in} = 2A_{out} + 2B_{out}$)	Z_out	Logic function realized
x_1	x_2					
0	0	0	0	0	0	x_1
0	1	0	0	0	0	
1	0	0	1	2	1	
1	1	1	0	2	1	

Hence the logic function realized by the whole network is $f(x_1, x_2) = x_1$.

6.4 THE PERCEPTRON

The perceptron is one of the earliest neural network models proposed by Rosenblatt in 1962. Early neural network enthusiasts were very fond of the perceptron due to its simple structure, pattern classifying behaviour, and learning ability. As far as the study of neural networks is concerned the perceptron is a very good starting point. This section provides the fundamental features of perceptron, namely, its structure, capability, limitations, and clues to overcome the limitations.

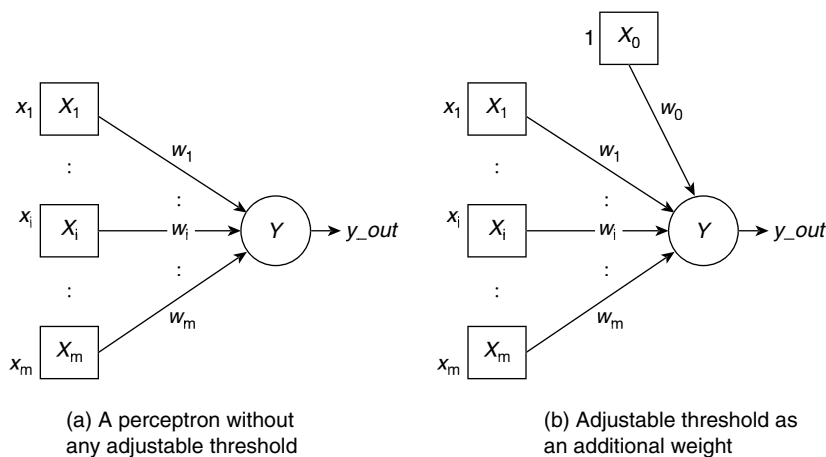


Fig. 6.17. Structure of a perceptron

6.4.1 The Structure

The structure of a perceptron is essentially same as that presented in subsection 6.1.2 and is shown here as Fig. 6.17(a). It consists of a number of input units $X_1, \dots, X_m$ and a processing unit Y . The connecting path from input X_i to Y is unidirectional and has a weight w_i . The weight w_i is positive if x_i is an excitatory input and is negative if it is inhibitive. The net input y_{in} of the Perceptron to Y is the algebraic sum of the weighted inputs.

$$y_in = \sum_{i=1}^m x_i w_i \quad (6.12)$$

Equation 6.12 can be expressed concisely in matrix notation as given in Equations 6.13(a) and 6.13(b).

$$y_in = \sum_{i=1}^m x_i w_i = [x_1 \ x_2 \ \dots \ x_m] \times \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix} \quad (6.13a)$$

$$\therefore y_in = X \times W^T \quad (6.13b)$$

Here $X = [x_1, \dots, x_m]$ and $W = [w_1, \dots, w_m]$ are the input and the weight vectors.

The perceptron sends an output 1 if the net input y_in is greater than a predefined adjustable threshold value θ , otherwise it sends output 0. Hence, the activation function of a perceptron is given by Equation 6.6, repeated below.

$$y_out = f(y_in) = \begin{cases} 1, & \text{if } y_in > \theta, \\ 0, & \text{if } y_in \leq \theta. \end{cases} \quad (6.14)$$

Therefore, in matrix notation,

$$y_out = \begin{cases} 1, & \text{if } X \times W^T > \theta, \\ 0, & \text{Otherwise.} \end{cases} \quad (6.15)$$

It is customary to include the adjustable threshold θ as an additional weight w_0 . This additional weight w_0 is attached to an input X_0 which is permanently maintained as 1. The modified structure inclusive of the adjustable weight w_0 and the additional input unit X_0 replacing the threshold θ is shown in Fig. 6.16(b). The expressions for the net input y_in and the activation y_out of the perceptron now take the forms

$$y_in = x_0 w_0 + x_1 w_1 + \dots + x_m w_m = \sum_{i=0}^m x_i w_i \quad (6.16)$$

and

$$y_out = f(y_in) = \begin{cases} 1, & \text{if } \sum_{i=0}^m x_i w_i > 0, \\ 0, & \text{otherwise.} \end{cases} \quad (6.17)$$

The following points should be noted regarding the structure of a perceptron.

- i) The inputs to a perceptron $x_0, \dots, x_m$ are real values.
- ii) The output is binary (0, or 1).
- iii) The perceptron itself is the totality of the input units, the weights, the summation processor, activation function, and the adjustable threshold value.

The perceptron acts as the basic ANN structure for pattern classification. The next subsection describes the capabilities of a perceptron as a pattern classifier.

6.4.2 Linear Separability

As mentioned earlier, perceptrons have the capacity to classify patterns. However, this pattern-classifying capacity is not unconditional. In this subsection, we investigate the criteria for a perceptron to act properly as a pattern classifier.

Let us consider, for example, two sets of points on a two-dimensional Cartesian plane $A = \{a, b, c\} = \{(-1, 3), (1, 2), (3, 3)\}$, and $B = \{p, q, r\} = \{(0, -1), (2, 1), (3, 0)\}$. These points are shown in Fig. 6.18. The points belonging to the set A, or B are indicated with white, and black dots respectively.

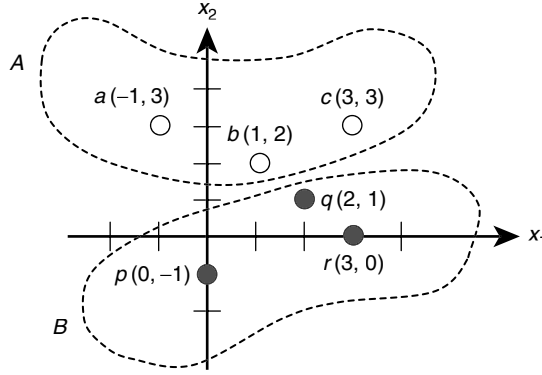


Fig. 6.18. A classification problem consisting for two sets of patterns A, and B.

We look for a classifier that can take an input pattern of the form (x_1, x_2) and decide to which class, A or B, the pattern belongs. A perceptron that carries out this job is shown in Fig. 6.19. The weights of the perceptron are carefully chosen so that the desired behaviour is achieved. However, we shall see that the weights need not to be *chosen*, but be *learnt* by the perceptron. The activation $y_{out} = 1$ if the input pattern belongs to class A, and $y_{out} = 0$ if it belongs to class B. Table 6.5 verifies that the perceptron shown in Fig. 6.19 can solve the classification problem posed in Fig. 6.18.

In fact, the pattern classifying capability of a perceptron is determined by the equations

$$y_{in} = 0, \quad (6.18)$$

$$\text{or, } \sum_{i=0}^m x_i w_i = 0, \quad (6.19)$$

$$\text{or, } x_0 w_0 + x_1 w_1 + \dots + x_m w_m = 0 \quad (6.20)$$

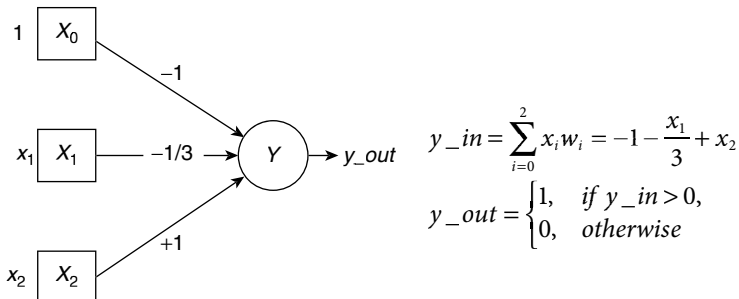


Fig. 6.19. A perceptron to solve the classification problem shown in Fig. 1.17.

Table 6.6. Details of Classification of Patterns Between Sets A and B

#		Input pattern		Net input (y_{in})	Activation (y_{out})	Class
		x_1	x_2			
1	a	-1	3	$2^{1/3}$	1	A
2	b	1	2	$2/3$	1	A
3	c	3	3	1	1	A
4	p	0	-1	-2	0	B
5	q	2	1	$-2/3$	0	B
6	r	3	0	-2	0	B

Applying Equations 6.18–6.20 for the perceptron under consideration we get

$$y_{in} = -1 - \frac{x_1}{3} + x_2 = 0$$

$$\text{or, } x_2 = \frac{x_1}{3} + 1 \quad (6.21)$$

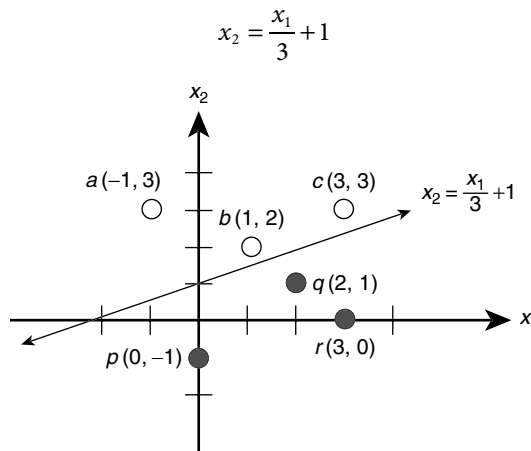
Equation 6.21 represents a straight line that separates the given sets of patterns A and B (Fig. 6.20). For two dimensional patterns of the form (x_1, x_2) the equation in terms of the weights looks like

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2} \quad (6.22)$$

Similarly, when the patterns are 3-dimensional and of the form (x_1, x_2, x_3) , Equation 6.20 would be

$$x_3 = -\frac{w_2}{w_3}x_2 - \frac{w_1}{w_3}x_1 - \frac{w_0}{w_3} \quad (6.23)$$

which represents a plane surface. In general, for n -dimensional patterns, Equation 6.20 represents a hyperplane in the corresponding n -dimensional hyperspace. Such a plane for a given perceptron capable of solving certain classification problem is known as a decision plane, or more generally, the decision surface.

**Fig. 6.20.** Linearly separable set of patterns

A linearly separable set of patterns is one that can be completely partitioned by a decision plane into two classes. The nice thing about the perceptrons is, for a given set of linearly separable patterns, it is always possible to find a perceptron that solves the corresponding classification problem. The only thing we have to ensure is to find the appropriate combination of values for the weights $w_0, w_1, \dots, w_m$. This is achieved through a process called learning or training by a perceptron. The famous perceptron convergence theorem (Rosenblatt [1962]) states that a perceptron is guaranteed to learn the appropriate values of the weights $w_0, w_1, \dots, w_m$ so that the given a set of linearly separable patterns are properly classified by the perceptron. There are various techniques for training neural networks. A brief overview of these techniques is presented in the later parts of this chapter.

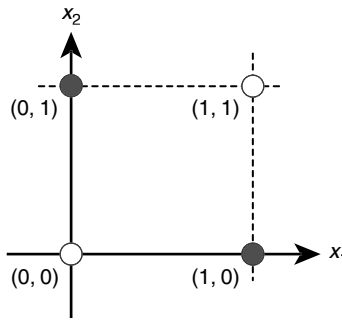


Fig. 6.21. The XOR problem offers patterns that are not linearly separable

6.4.3 The XOR Problem

Real life classification problems, however, rarely offer such well-behaved linearly separable data as required by a perceptron. Minsky and Papert [1969] showed that no perceptron can learn to compute even a trivial function like a two bit XOR. The reason is, there is no single straight line that may separate the 1-producing patterns $\{(0, 1), (1, 0)\}$ from the patterns 0-producing patterns $\{(0, 0), (1, 1)\}$. This is illustrated in Fig. 6.21. Is it possible to overcome this limitation? If yes, then how? There are two ways. One of them is to draw a curved decision surface between the two sets of patterns as shown in Fig. 6.22(a) or 6.22(b). However, perceptron can not model any curved surface.

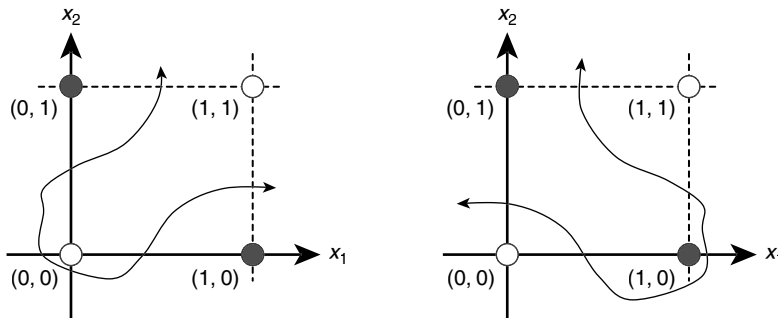


Fig. 6.22. Solving the XOR problem with curved decision surface

The other way is to employ two, instead of one, decision lines. The first line isolates the point (1, 1) from the other three points, viz., (0, 0), (0, 1), and (1, 0). The second line partitions $\{(0, 0), (0, 1), (1, 0)\}$ into

the classes $\{(0, 1), (1, 0)\}$ and $\{(0, 0)\}$. The technique is shown in Fig. 6.23. Fig. 6.23(b) shows another pair of lines that solve the problem in a different way.

Using this idea, it is possible design a multi-layered perceptron to solve the XOR problem. Such a multilayered perceptron is shown in Fig. 6.24. Here the first perceptron Y fires only when the input is $(1, 1)$. But this sends a large inhibitive signal of magnitude -2.0 to the second perceptron Z so that the excitatory signals from x_1 and x_2 to Z are overpowered. As a result the net input to Z attains a negative value and the perceptron fails to fire. On the other hand, the remaining three input patterns, $(0, 0)$, $(0, 1)$, and $(1, 0)$, for which perceptron Y does not influence Z , are processed by Z in the desired way.

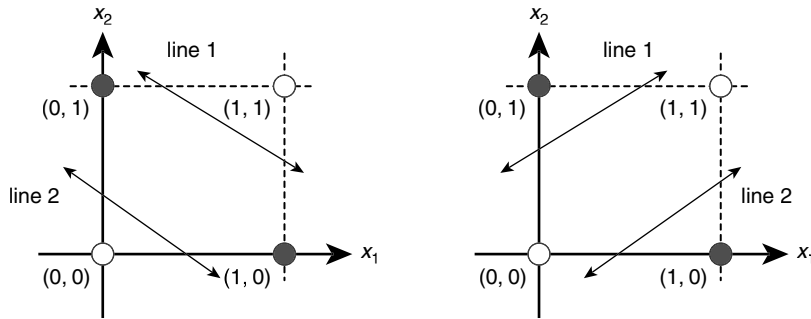


Fig. 6.23. Solving the XOR problem with two decision lines

Hence, the arrangement of perceptrons shown in Fig. 6.24 successfully classifies the patterns posed by the XOR problem. The critical point is, the perceptron convergence theorem is no longer applicable to multilayer perceptrons. The perceptron learning algorithm can adjust the weights of the interconnections between the inputs and the processing element, but not between the processing elements of two different perceptrons. The weight -2.0 in Fig. 6.24 is decided through observation and analysis, not by training.

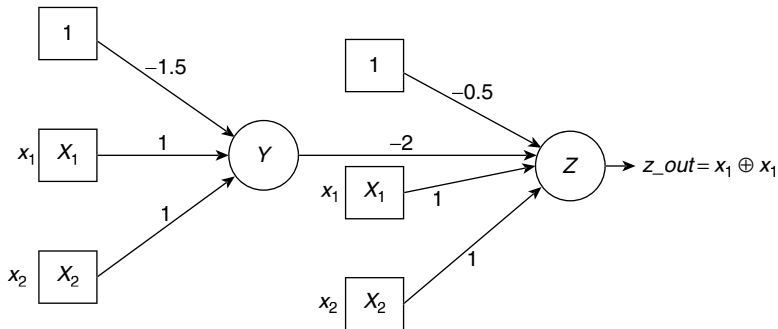


Fig. 6.24. A multi-layer Perceptron to solve the XOR problem

6.5 NEURAL NETWORK ARCHITECTURES

An ANN consists of a number of artificial neurons connected among themselves in certain ways. Some-time these neurons are arranged in layers, with interconnections across the layers. The network may, or may not, be fully connected. Moreover, the nature of the interconnection paths also differs. They are

either unidirectional, or bidirectional. The topology of an ANN, together with the nature of its interconnection paths, is generally referred to as its architecture. This section presents an overview of the major ANN architectures.

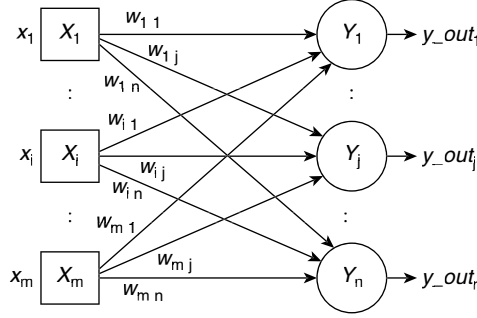


Fig. 6.25. Structure of a single-layer feed forward ANN

6.5.1 Single Layer Feed Forward ANNs

Single layer feed forward is perhaps the simplest ANN architecture. As shown in Fig. 6.25, it consists of an array of input neurons connected to an array of output neurons. Since the input neurons do not exercise any processing power, but simply forward the input signals to the subsequent neurons, they are not considered to constitute a layer. Hence, the only layer in the ANN shown in Fig. 6.25 is composed of the output neurons $Y_1, \dots, Y_n$.

The ANN shown in Fig. 6.25 consists of m inputs $X_1, \dots, X_m$ and n outputs $Y_1, \dots, Y_n$. The signal transmitted by input X_i is denoted as x_i . Each input X_i is connected to each output Y_j . The weight associated with the path between X_i and Y_j is denoted as w_{ij} . The interconnection paths are unidirectional and are directed from the input units to the output units.

The net input y_in_1 to the output unit Y_1 is given by

$$y_in_1 = x_1 w_{11} + x_2 w_{21} + \dots + x_m w_{m1} = \sum_{i=1}^m x_i w_{i1} \quad (6.24)$$

In vector notation, Equation 6.24 can be expressed as

$$y_in_1 = [x_1 \dots x_m] \times \begin{bmatrix} w_{11} \\ \vdots \\ w_{m1} \end{bmatrix} = X \times W_{*1} \quad (6.25)$$

where $X = [x_1 \dots x_m]$ is the input vector of the input signals and W_{*1} is the first column of the weight matrix

$$W = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \dots & \vdots \\ w_{m1} & w_{m2} & \dots & w_{mn} \end{bmatrix} \quad (6.26)$$

In general, the net input y_in_j to the output unit Y_j is given by

$$y_in_j = [x_1 \dots x_m] \times \begin{bmatrix} w_{1j} \\ \vdots \\ w_{mj} \end{bmatrix} = X \times W_{*j} \quad (6.27)$$

If Y_in denotes the vector for the net inputs to the array of output units

$$Y_in = [y_in_1 \dots y_in_n]$$

then the net input to the entire array of output units can be expressed concisely in matrix notation as

$$Y_in = X \times W \quad (6.28)$$

The signals transmitted by the output units, of course, depend on the nature of the activation functions. There are various kinds of activation functions and these are discussed in greater details in the next subsection. The basic single layer feed forward network architecture presented in this section has its own variations. For example, in some cases the network may allow interconnections among the input or output units themselves. These will be discussed later in greater details.

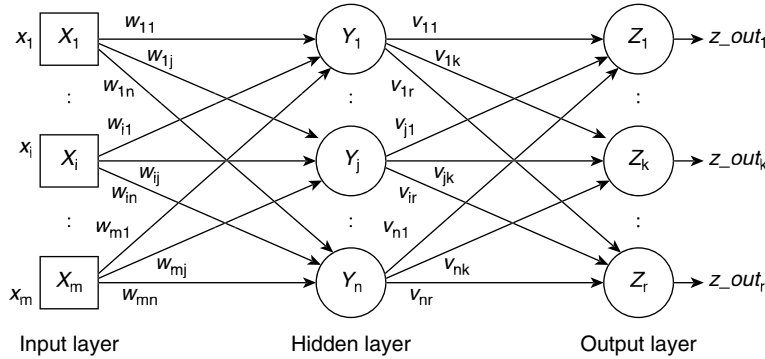
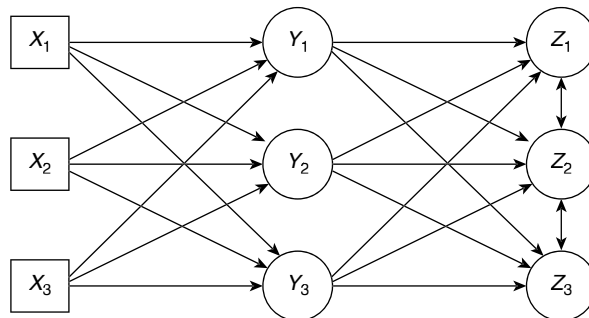


Fig. 6.26. A multi-layer feed forward network with one hidden layer

6.5.2 Multilayer Feed Forward ANNs

A multilayer feed forward net is similar to single layer feed forward net except that there is (are) one or more additional layer(s) of processing units between the input and the output layers. Such additional layers are called the hidden layers of the network. Fig. 6.26 shows the architecture of a multi-layer feed forward neural net with one hidden layer.



(a) Competitive ANN with locally connected output units

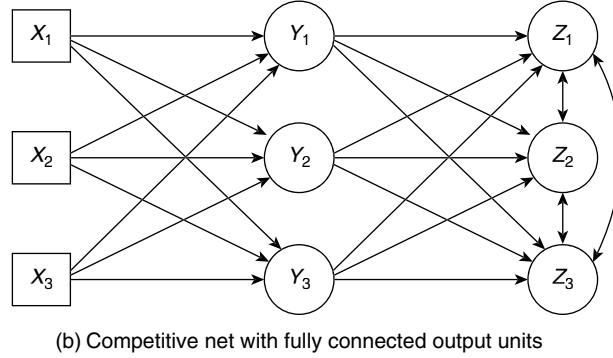


Fig. 6.27. Competitive ANN architectures

The expressions for the net inputs to the hidden layer units and the output units are obtained as

$$Y_in = X \times W \quad (6.29)$$

$$Z_in = Y_out \times V \quad (6.30)$$

where

$X = [x_1, x_2, \dots, x_m]$, X is the input vector,

$Y_in = [y_in_1, y_in_2, \dots, y_in_n]$, is the net input vector to the hidden layer,

$Z_in = [z_in_1, z_in_2, \dots, z_in_r]$, is the net input vector to the output layer,

$Y_out = [y_out_1, y_out_2, \dots, y_out_n]$, is the output vector from the hidden layer,

W and V are the weight matrices for the interconnections between the input layer, hidden layer, and output layer respectively.

$$W = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \dots & \vdots \\ w_{m1} & w_{m2} & \dots & w_{mn} \end{bmatrix} \text{ and } V = \begin{bmatrix} v_{11} & v_{12} & \dots & v_{1r} \\ v_{21} & v_{22} & \dots & v_{2r} \\ \vdots & \vdots & \dots & \vdots \\ v_{n1} & v_{n2} & \dots & v_{nr} \end{bmatrix}$$

Fig. 6.26 shows the structure of a multi-layered feed forward net with one hidden layer. Obviously, it is possible to include more than one hidden layers in such networks.

6.5.3 Competitive Network

Competitive networks are structurally similar to single layer feed forward nets. However, the output units of a competitive neural net are connected among themselves, usually through negative weights. Fig. 6.27(a) and 6.27(b) show two kinds of competitive networks. In Fig. 6.27(a), the output units are connected only to their respective neighbours, whereas the network of Fig. 6.27(b) shows a competitive network whose output units are fully connected. For a given input pattern, the output units of a competitive network tend to compete among themselves to represent that input. Thus the name competitive network.

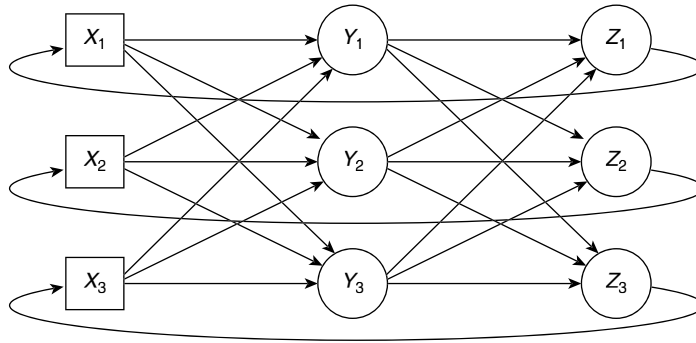


Fig. 6.28 A recurrent network with feedback paths from the output layer to the input layer

6.5.4 Recurrent Networks

In a feed forward network signals flow in one direction only and that is from the input layer towards the output layer through the hidden layers, if any. Such networks do not have any feedback loop. In contrast, a recurrent network allows feedback loops. A typical recurrent network architecture is shown in Fig. 6.28. Fig. 6.29(a) shows a fully connected recurrent network. Such networks contain a bidirectional path between every pair of processing elements. Moreover, a recurrent network may contain self loops, as shown in Fig. 6.29(b).

6.6 ACTIVATION FUNCTIONS

The output from a processing unit is termed as its activation. Activation of a processing unit is a function of the net input to the processing unit. The function that maps the net input value to the output signal value, i.e., the activation, is known as the activation function of the unit. Some common activation functions are presented below.

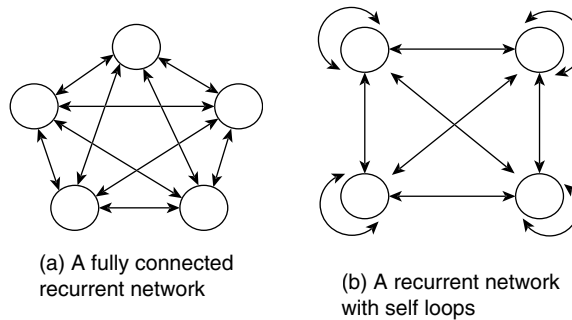


Fig. 6.29. Recurrent network architectures

6.6.1 Identity Function

The simplest activation function is the identity function that passes on the incoming signal as the outgoing signal without any change. Therefore, the identity activation function $g(x)$ is defined as

$$g(x) = x \quad (6.31)$$

Fig. 6.30 shows the form of the identity function graphically. Usually, the units of the input layer employ the identity function for their activation. This is because in ANN, the role of an input unit is to forward the incoming signal as it is to the units in the next layer through the respective weighted paths.

6.6.2 Step function

Another frequently used activation function is the step function. The basic step function produces a 1 or 0 depending on whether the net input is greater than 0 or otherwise. This is the only activation function we have used so far in this text. Mathematically the step function is defined as follows.

$$g(x) = \begin{cases} 1, & \text{if } x > 0, \\ 0, & \text{otherwise.} \end{cases} \quad (6.32)$$

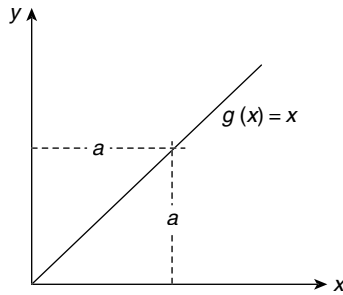
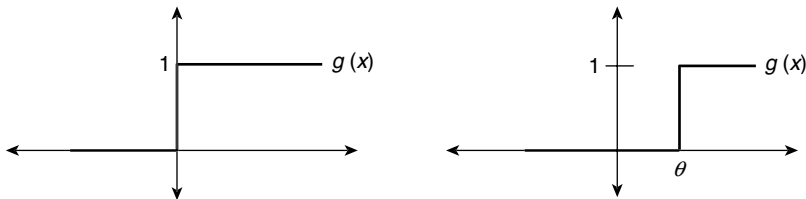


Fig. 6.30. The identity activation function

Fig. 6.31(a) shows the shape of the basic step function graphically. Occasionally, instead of 0 a non-zero threshold value θ is used. This is known as the threshold function and is defined as

$$g(x) = \begin{cases} 1, & \text{if } x > \theta, \\ 0, & \text{otherwise.} \end{cases} \quad (6.33)$$



(a) Basic Step function

(b) Threshold function

Fig. 6.31. Step functions

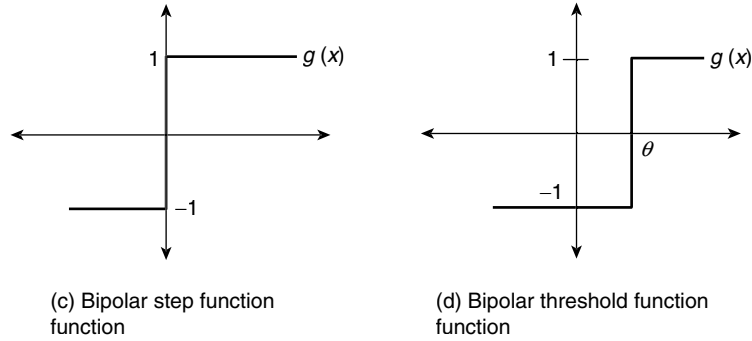


Fig. 6.31. (Continued)

The shape of the threshold function is shown in Fig. 6.31(b). The step function is also known as the heaviside function. The step functions discussed so far are binary step functions since they always evaluate to 0 or 1.

Occasionally, it is more convenient to work with bipolar data, -1 and $+1$, than the binary data. If a signal of value 0 is sent through a weighted path, the information contained in the interconnection weight is lost as it is multiplied by 0. To overcome this problem, the binary input is converted to bipolar form and then a suitable bipolar activation function is employed. Accordingly, binary step functions have their bipolar versions. The output of a bipolar step function is -1 , or $+1$, not 0, or 1. The bipolar step function and threshold function are shown in Fig. 1.31(c) and (d) respectively. They are defined as follows.

Bipolar step function:

$$g(x) = \begin{cases} +1, & \text{if } x > 0, \\ -1, & \text{otherwise.} \end{cases} \quad (6.34)$$

Bipolar threshold function:

$$g(x) = \begin{cases} +1, & \text{if } x > \theta, \\ -1, & \text{otherwise.} \end{cases} \quad (6.35)$$

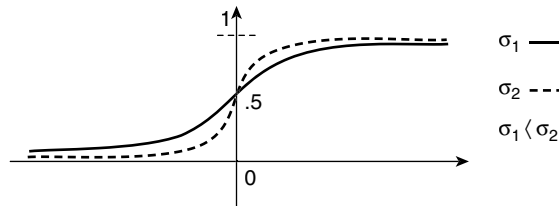


Fig. 6.32. Binary Sigmoid function

6.6.3 The Sigmoid Function

As the step function is not continuous it is not differentiable. Some ANN training algorithm requires that the activation function be continuous and differentiable. The step function is not suitable for such cases. Sigmoid functions have the nice property that they can approximate the step function to the

desired extent without losing its differentiability. Binary sigmoid, also referred to as the logistic sigmoid, is defined by Equation 6.36.

$$g(x) = \frac{1}{1 + e^{-\sigma x}} \quad (6.36)$$

The parameter σ in Equation 6.36 is known as the steepness parameter. The shape of the sigmoid function is shown in Fig. 6.32. The transition from 0 to 1 could be made as steep as desired by increasing the value of σ to appropriate extent.

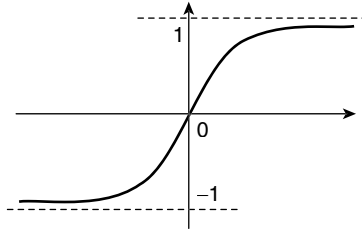


Fig. 6.33. Bipolar Sigmoid function

The first derivative of $g(x)$, denoted by $g'(x)$ is expressed as

$$g'(x) = \sigma g(x)(1 - g(x)) \quad (6.37)$$

Depending on the requirement, the binary sigmoid function can be scaled to any range of values appropriate for a given application. The most widely used range is from -1 to $+1$, and the corresponding sigmoid function is referred to as the bipolar sigmoid function. The formulae for the bipolar sigmoid function and its first derivative are given below as Equations 6.38 and 6.39 respectively. Fig. 1.33 presents its form graphically.

$$g(x) = \frac{1 - e^{-\sigma x}}{1 + e^{-\sigma x}} \quad (6.38)$$

$$g'(x) = \frac{\sigma}{2} (1 + g(x))(1 - g(x)) \quad (6.39)$$

6.6.4 Hyperbolic Tangent Function

Another bipolar activation function that is widely employed in ANN applications is the hyperbolic tangent function. The function, as well as its first derivative, are expressed by Equations 6.40 and 6.41 respectively.

$$h(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (6.40)$$

$$h'(x) = (1 + h(x))(1 - h(x)) \quad (6.41)$$

The hyperbolic tangent function is closely related to the bipolar sigmoid function. When the input data is binary and not continuously valued in the range from 0 to 1, they are generally converted to bipolar form and then a bipolar sigmoid or hyperbolic tangent activation function is applied on them by the processing units.

6.7 LEARNING BY NEURAL NETS

An ANN is characterized by three entities, its architecture, activation function, and the learning technique. Learning by an ANN refers to the process of finding the appropriate set of weights of the interconnections so that the ANN attains the ability to perform the designated task. The process is also referred to as training the ANN.

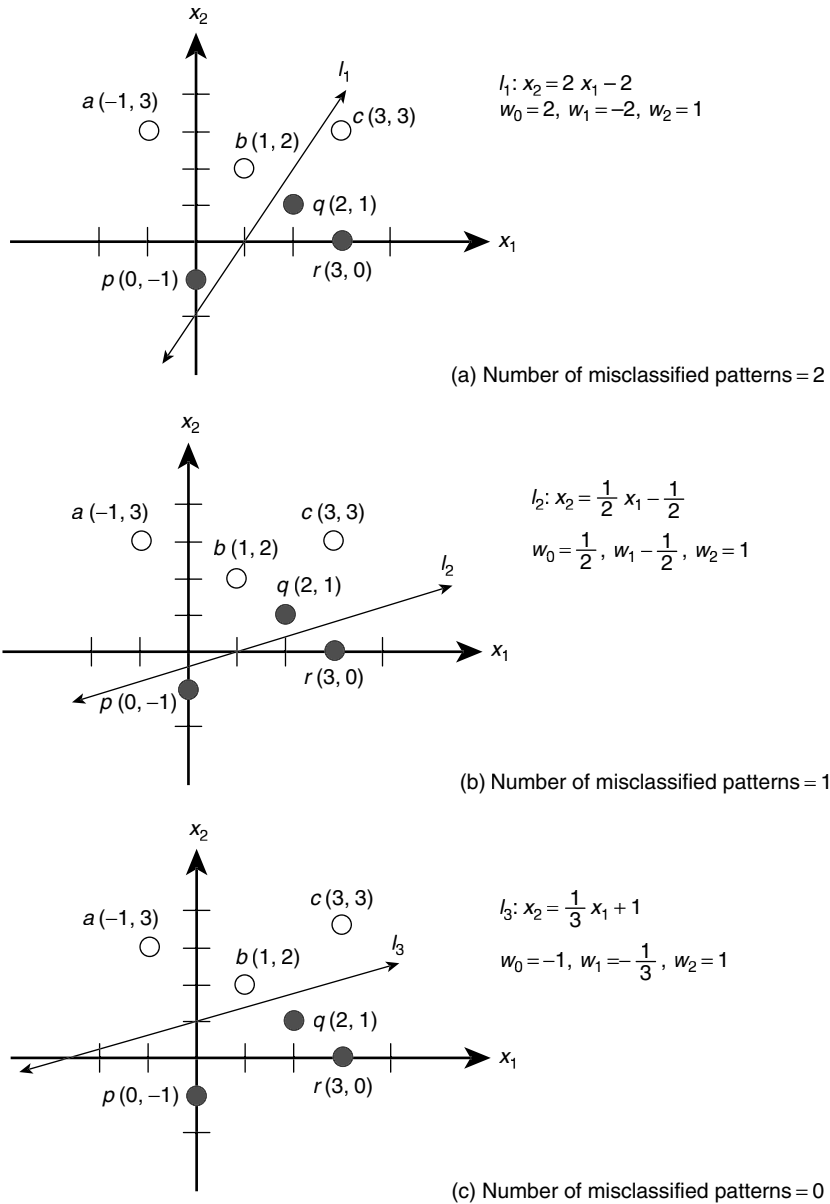


Fig. 6.34. Learning a pattern classification task by a perceptron

Let us, once again, consider the classification problem posed in Section 6.4.2 (Fig. 6.18). A perceptron is presented (Fig. 6.19) to solve the problem. It has the following combination of interconnection weights : $w_0 = -1$, $w_1 = -1/3$, $w_2 = 1$. It is easy to verify that an arbitrary set of weights may not be able to solve the given classification problem. The question is, how to find the appropriate set of weights for an ANN so that the ANN is able to solve a given problem? One way is to start with a set of weights and then gradually modify them to arrive at the final set of weights. This is illustrated in Fig. 6.34(a), (b), (c). Suppose we start with a set of randomly chosen values, say $w_0 = 2$, $w_1 = -2$, $w_2 = 1$. The corresponding decision line is l_1 (Fig. 6.34(a)) which is algebraically expressed by the equation $x_2 = 2x_1 - 2$. As Fig. 6.34(a) shows, line l_1 classifies the points a $(-1, 3)$, b $(1, 2)$, q $(2, 1)$, and r $(3, 0)$ correctly. But it misclassifies the points c $(3, 3)$ (wrongly put in the class B) and the point p $(0, -1)$ (wrongly put in the class A). In the next step the weights are modified to $w_0 = 1/2$, $w_1 = -1/2$, $w_2 = 1$, so that the new decision line $l_2 : x_2 = 1/2 x_1 - 1/2$ reduces the number of misclassified data to 1, only the point q $(2, 1)$. This is shown in Fig. 6.34(b). Then the weights are further modified to obtain the decision line $l_3 : x_2 = 1/3 x_1 + 1$, that leaves no pattern misclassified (Fig. 6.34(c)).

This learning instance illustrates the basic concept of supervised learning, i.e., learning assisted by a teacher. However, quite a number of issues are yet to be addressed. For example, given a set of interconnection weights, how to determine the adjustments required to compute the next set of weights? Moreover, how do we ensure that the process converges, i.e., the number of misclassified patterns are progressively reduced and eventually made 0? The subsequent parts of this section briefly introduce the popular learning algorithms employed in ANN systems.

The basic principle of ANN learning is rather simple. It starts with an initial distribution of interconnection weights and then goes on adjusting the weights iteratively until some predefined stopping criterion is satisfied. Therefore, if $w(k)$ be the weight of a certain interconnection path at the k th iteration, then $w(k+1)$, the same at the $(k+1)$ th iteration, is obtained by

$$w(k+1) = w(k) + \Delta w(k) \quad (6.42)$$

where $\Delta w(k)$ is the k th adjustment to weight w . A learning algorithm is characterized by the method undertaken by it to compute $\Delta w(k)$.

6.7.1 Supervised Learning

A neural network is trained with the help of a set of patterns known as the training vectors. The outputs for these vectors might be, or might not be, known beforehand. When these are known, and that knowledge is employed in the training process, the training is termed as *supervised* learning. Otherwise, the learning is said to be *unsupervised*. Some popular supervised learning methods are perceptron learning, delta learning, least-mean-square (LMS) learning, correlation learning, outstar learning etc. These are briefly introduced below.

(a) Hebb Rule

The Hebb rule is one of the earliest learning rules for ANNs. According to this rule the weight adjustment is computed as

$$\Delta w_i = x_i \times t \quad (6.49)$$

where t is the target activation.

There are certain points to be kept in mind regarding the Hebb learning rule. First, Hebb rule can not learn when the target is 0. This is because the weight adjustment Δw_i becomes zero when $t = 0$,

irrespective of the value of x_i . Hence, obviously, the Hebb rule results in better learning if the input / output both are in bipolar form. The most striking limitation of the Hebb rule is it does not guarantee to learn a classification instance even if the classes are linearly separable. This is illustrated in Chapter 7. The Example 6.6 below gives an instance of Hebb Learning.

Example 6.6 (Realizing the logical AND function through Hebb learning)

To realize a two input AND function we need a net with two input units and one output unit. A bias is also needed. Hence the structure of the required neural net should be as shown in Fig. 6.35. Moreover, the input and output signals must be in *bipolar* form, rather than the *binary* form, so that the net may be trained properly. The advantage of bipolar signals over binary signals is discussed in greater detail in Chap. 7. Considering the truth table of AND operation, and the fact that the bias is permanently set to 1, we get the training set depicted in Table 6.7.

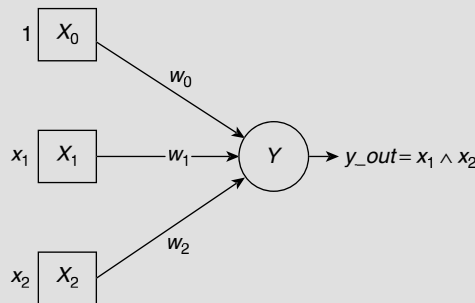


Fig. 6.35. Structure of a neural net to realize the AND function

Table 6.7 Training set for AND function

Input Patterns			Output
x_0	x_1	x_2	t
+1	1	1	1
+1	1	-1	-1
+1	-1	1	-1
+1	-1	-1	-1

During the training process, all weights are initialized to 0. Therefore initially

$$w_0 = w_1 = w_2 = 0$$

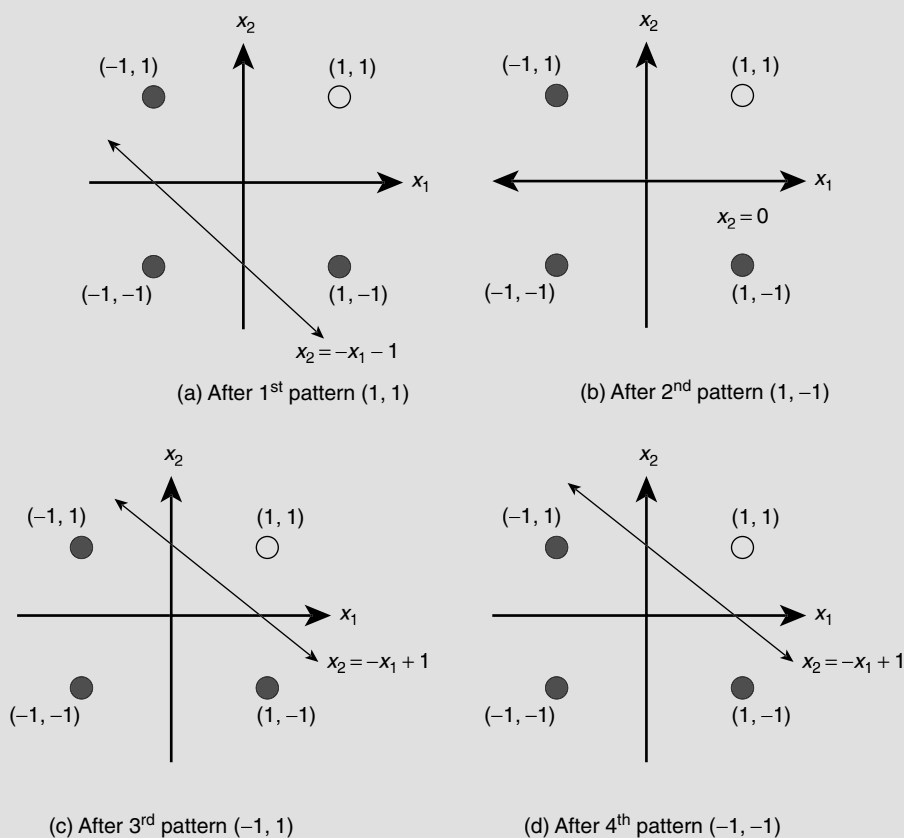
At each training instance, the weights are changed according to the formula

$$w_i(\text{new}) = w_i(\text{old}) + \Delta w_i$$

where Δw_i the increment in w_i is computed as $\Delta w_i = x_i \times t$. After initialization, the progress of the learning process by the network is shown in Table 6.8.

Table 6.8. Hebbian learning of AND function

#	Training Pattern			Target output	Weight Adjustments			Weights		
	x_0	x_1	x_2		Δw_0	Δw_1	Δw_2	w_0	w_1	w_2
0								0	0	0
1	1	1	1	1	1	1	1	1	1	1
2	1	1	-1	-1	-1	-1	1	0	0	2
3	1	-1	1	-1	-1	1	-1	-1	1	1
4	1	-1	-1	-1	-1	1	1	-2	2	2

**Fig. 6.36.** Hebbian training to learn AND function

Hence, on completion of one epoch of training, the weight vector is $W = [w_0, w_1, w_2] = [-2, 2, 2]$. The progress in learning by the net can be visualized by observing the orientation of the decision line after each training instance. Putting the values of the interconnection weights in the equation

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2}$$

- we get
- (i) $x_2 = -x_1 - 1$, after the 1st training instance,
 - (ii) $x_2 = 0$, after the 2nd training instance,
 - (iii) $x_2 = -x_1 + 1$, after the 3rd training instance, and finally,
 - (iv) $x_2 = -x_1 + 1$, after the 4th training instance.

This progress in the learning process is depicted in Fig. 6.36(a)-(d). We see that after training with the first pattern $[x_0, x_1, x_2] = [1, 1, 1]$, the ANN learns to classify two patterns $(-1, -1)$ and $(1, 1)$ successfully. But it fails to classify correctly the other two patterns $(-1, 1)$ and $(1, -1)$. After learning with the second pattern $(1, -1)$ the situation is better. Only $(-1, 1)$ is still misclassified. This is corrected after training with the third pattern $(-1, 1)$. Now all the -1 producing patterns are, i.e., $(1, -1)$, $(-1, 1)$ and $(-1, -1)$ are in the same class and the remaining pattern $(1, 1)$ constitutes the other class.

(b) Perceptron Learning Rule

Let us consider a simple ANN consisting of a single perceptron Y with $m+1$ input units $X_0, \dots, X_m$ as shown in Fig. 6.17(b). The corresponding weights of the interconnections are $w_0, \dots, w_m$. The bias is introduced as the weight w_0 connected to the input X_0 whose activation is fixed at 1. For the output unit, it is convenient to use the bipolar activation function :

$$y_{out} = \begin{cases} 1, & \text{if } y_{in} > 0 \\ 0, & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases} \quad (6.43)$$

Now, let $X = (x_0, \dots, x_m)$ be a training vector for which the output of the perceptron is expected to be t , where $t = 1, 0$, or -1 . The current combination of the weights is given by the weight vector $W = (w_0, \dots, w_m)$. If the perceptron produces the desired output, then the weights $w_0, \dots, w_m$ need not be changed and they are to be kept unaltered. If, however, the perceptron misclassifies X negatively (meaning, it erroneously produces -1 instead of the desired output $+1$) then the weights should be appropriately increased. Conversely, the weights are to be decreased in case the perceptron misclassifies X positively (i.e., it erroneously produces $+1$ instead of the desired output -1). The learning strategy of the perceptron is summarized in Table 6.9.

Hence the perceptron learning rule is informally stated as

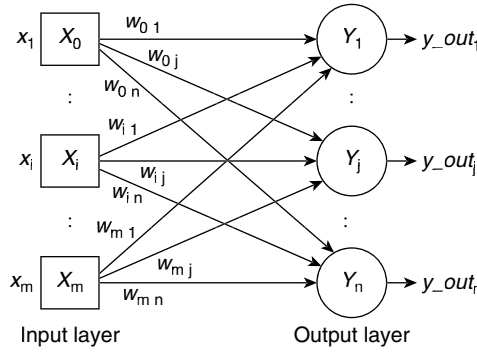
IF the output is erroneous THEN adjust the interconnection weights
ELSE leave the interconnection weights unchanged.

In more precise terms,

IF $y_{out} \neq t$ THEN
FOR $i = 0$ TO m DO $w_i(new) = w_i(old) + \eta \times t \times x_i$
ELSE
FOR $i = 0$ TO m DO $w_i(new) = w_i(old)$.

Table 6.9. Perceptron learning rules

#	Condition	Action
1	The perceptron classifies the input pattern correctly. ($y_{out} = t$)	No change in the current set of weights $w_0, w_1, \dots, w_m$.
2	The perceptron misclassifies the input pattern negatively. ($y_{out} = -1$, but $t = +1$)	Increase each w_i by Δw_i , where Δw_i is proportional to x_i , for all $i = 0, 1, \dots, m$.
3	The perceptron misclassifies the input pattern positively, ($y_{out} = +1$, but $t = -1$)	Decrease each w_i by Δw_i , where Δw_i is proportional to x_i , for all $i = 0, 1, \dots, m$.

**Fig. 6.37.** Structure of an ANN with several Perceptrons

Thus the perceptron learning rule can be formulated as

$$\Delta w_i = \eta \times (t - y_{out}) \times x_i, \text{ for } i = 0, 1, \dots, m \quad (6.44)$$

Here η is a constant known as the learning rate. It should be noticed that when a training vector is correctly classified then $y_{out} = t$, and the weight adjustment $\Delta w_i = 0$. When $y_{out} = -1$ but $t = +1$, i.e., the pattern is misclassified negatively, then $t - y_{out} = +2$ so that Δw_i is incremental and is proportional to x_i . If, however, the input pattern is misclassified positively, the adjustment is decremental, and obviously, proportional to x_i . Using matrix notation, the perceptron learning rule may now be written as

$$\Delta W = \eta \times (t - y_{out}) \times X \quad (6.45)$$

where, ΔW and X are the vectors corresponding to the interconnection weights and the inputs.

$$\Delta W = [\Delta w_0, \dots, \Delta w_m], \text{ and}$$

$$X = [x_0, x_1, \dots, x_m].$$

Equation 6.45 can be easily extended to a network of several perceptrons at the output layer. Such architecture is shown in Fig. 6.37. The net inputs to the perceptrons are

$$\begin{bmatrix} y_{in1} \\ \vdots \\ y_{inj} \\ \vdots \\ y_{inn} \end{bmatrix} = \begin{bmatrix} w_{01} & \cdots & w_{i1} & \cdots & w_{m1} \\ \vdots & & \vdots & & \vdots \\ w_{0j} & \cdots & w_{ij} & \cdots & w_{mj} \\ \vdots & & \vdots & & \vdots \\ w_{0n} & \cdots & w_{in} & \cdots & w_{mn} \end{bmatrix} \times \begin{bmatrix} x_0 \\ \vdots \\ x_i \\ \vdots \\ x_m \end{bmatrix}$$

or,

$$Y_in^T = W^T \times X^T \quad (6.46)$$

For such an ANN, the adjustment Δw_{ij} of the weight w_{ij} is given by

$$\Delta w_{ij} = \eta \times (t_j - y_out_j) \times x_i \quad (6.47)$$

Let us assume the following matrix notations :

$$\Delta W = \begin{bmatrix} \Delta w_{01} & \Delta w_{02} & \cdots & \Delta w_{0n} \\ \Delta w_{11} & \Delta w_{12} & \cdots & \Delta w_{1n} \\ \vdots & \vdots & \vdots & \vdots \\ \Delta w_{m1} & \Delta w_{m2} & \cdots & \Delta w_{mn} \end{bmatrix}, T = [t_1 \quad \cdots \quad t_n],$$

$$X = [x_0, \quad \cdots, \quad x_m], \text{ and } Y_out = [y_out_1, \quad \cdots, \quad y_out_n].$$

Then the expression of the perceptron learning rule for the architecture shown in Fig. 6.34 becomes

$$[\Delta W]^T = \eta \times [T - Y_out]^T \times X \quad (6.48)$$

Example 6.7 (Learning the logical AND function by a perceptron)

Let us train a perceptron to realize the logical AND function. The training patterns and the corresponding target outputs for AND operation where the input and outputs are in bipolar form are given in Table 6.6. The structure of the perceptron is same as shown in Fig. 6.35. Activation function for the output unit is :

$$y_out = g(y_in) = \begin{cases} 1, & \text{if } y_in > 0 \\ 0, & \text{if } y_in = 0 \\ -1, & \text{if } y_in < 0 \end{cases}$$

The successive steps of computation with the first training pair are given below.

- i) Initialize weights : $w_0 = w_1 = w_2 = 0$, and the learning rate $\eta = 1$.
- ii) For the training pair $s : t = (1, 1, 1) : (1)$, compute the net input y_in and the activation y_out :

$$\begin{aligned} y_in &= w_0 \times x_0 + w_1 \times x_1 + w_2 \times x_2 \\ &= 0 \times 1 + 0 \times 1 + 0 \times 1 \\ &= 0 \\ \therefore y_out &= 0. \end{aligned}$$

- iii) Apply the perceptron learning rule to find the weight adjustments. The rule is :

If $y_out = t$, then the weights are not changed, otherwise change the weights according to the formula $w_i (new) = w_i (old) + \Delta w_i$ where $\Delta w_i = \eta t x_i$. In the present case, $y_out = 0$, and $t = 1$, and $y_out \neq t$. Therefore the weights are to be adjusted.

$$\begin{aligned} w_0 (new) &= w_0 (old) + \Delta w_0 \\ &= w_0 (old) + \eta t x_0 \\ &= 0 + 1 \times 1 \times 1 \\ &= 1. \end{aligned}$$

Similarly,

$$\begin{aligned}w_1(\text{new}) &= w_1(\text{old}) + \Delta w_1 \\&= w_1(\text{old}) + \eta t x_1 \\&= 0 + 1 \times 1 \times 1 \\&= 1.\end{aligned}$$

And,

$$w_2(\text{new}) = 1.$$

Putting these values of the weights in the formula

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2}$$

we get the separating line as $x_2 = -x_1 - 1$, which classifies the patterns (1, 1) and (-1, -1) correctly but misclassifies the patterns (-1, 1) and (1, -1) (Fig. 6.36(a)). A trace of the remaining steps of the first *epoch* of training (an *epoch* of training consists of successive application of each and every training pair of the training set for once) are noted in Table 6.10.

Hence, the set of weights obtained at the end of the first epoch of training is (-1, 1, 1) which represents to the decision line

$$x_2 = -x_1 + 1$$

It can be easily verified that this line (and the corresponding set of weights) successfully realizes the AND function for all possible input combinations. Hence, there is no need of further training.

Table 6.10. Perceptron learning of AND function

#	Input Pattern			Net input	Activation	Target output	Weight Adjustments			Weights		
	x_0	x_1	x_2	y_{in}	y_{out}	t	Δw_0	Δw_1	Δw_2	w_0	w_1	w_2
0										0	0	0
1	1	1	1	0	0	1	1	1	1	1	1	1
2	1	1	-1	1	1	-1	-1	-1	1	0	0	2
3	1	-1	1	2	1	-1	-1	1	-1	-1	1	1
4	1	-1	-1	-3	-1	-1	0	0	0	-1	1	1

(c) Delta / LMS (Least Mean Square), or, Widrow-Hoff Rule

Least Mean Square (*LMS*), also referred to as the Delta, or Widrow-Hoff Rule, is another widely used learning rule in ANN literature. Here the weight adjustment is computed as

$$\Delta w_i = \eta \times (t - y_{in}) \times x_i \quad (6.49)$$

where the symbols have their usual meanings. In *LMS* learning, the identity function is used as the activation function during the training phase. The learning rule minimizes mean squared error between the activation and the target value. The output of *LMS* learning is in binary form. Example 6.8 illustrates *LMS* learning of the AND function.

Example 6.8 (Realize the logical AND function through LMS learning rule)

The training patterns and the corresponding target outputs for AND operation where the input and outputs are in bipolar form are given in Table 6.7. The structure of the perceptron is same as shown in Fig. 6.35. The successive steps of computation for the first epoch are given below.

- (i) Initialize weights : $w_0 = w_1 = w_2 = .3$, and the learning rate $\eta = .2$ (These values are randomly chosen)
- (ii) For the training pair $s : t = (1, 1, 1) : (1)$, compute the net input y_in as

$$\begin{aligned} y_in &= w_0 \times x_0 + w_1 \times x_1 + w_2 \times x_2 \\ &= .3 \times 1 + .3 \times 1 + .3 \times 1 \\ &= .9 \end{aligned}$$

- (iii) Apply the LMS learning rule to find the weight adjustments. The rule is, $w_i (new) = w_i (old) + \Delta w_i$ where $\Delta w_i = \eta(t - y_in) x_i$. Hence

$$w_i (new) = w_i (old) + \eta(t - y_in) x_i,$$

In the present case, $y_in = .9$, and $t = 1$. Therefore

$$\begin{aligned} w_0 (new) &= w_0 (old) + \Delta w_0 \\ &= w_0 (old) + \eta(t - y_in) x_0 \\ &= .3 + .2 \times (1 - .9) \times 1 \\ &= .3 + .02 \\ &= .32 \end{aligned}$$

Similarly,

$$w_1 (new) = w_2 (new) = .32$$

Second iteration is to be carried out with the training pair $s : t = (1, 1, -1) : (-1)$. The net input y_in is now

$$\begin{aligned} y_in &= w_0 \times x_0 + w_1 \times x_1 + w_2 \times x_2 \\ &= .32 \times 1 + .32 \times 1 + .32 \times (-1) \\ &= .32 \end{aligned}$$

The new weights are computed as follows :

$$\begin{aligned} w_0 (new) &= w_0 (old) + \Delta w_0 \\ &= w_0 (old) + \eta(t - y_in) x_0 \\ &= .32 + .2 \times (-1 - .32) \times 1 \\ &= .32 - .264 \\ &= .056 \end{aligned}$$

Similarly, $w_1(\text{new}) = .056$

However, $w_2(\text{new}) = .32 + .2 \times (-1 - .32) \times (-1)$
 $= .32 + .264$
 $= .584$

Hence, at the end of the second iteration, we get $w_0 = .056$, $w_1 = .056$, and $w_2 = .584$. The details of the computation for the first epoch are recorded in Table 6.11. We see that the weights arrived at the end of the first epoch are $w_0 = -.281$, $w_1 = .393$, and $w_2 = .287$. Table 6.12 shows that this set of weights is appropriate to realize the *AND* function. Hence there is no need to iterate further.

Table 6.11. *LMS Learning of AND function*

#	Input Pattern			Net input	Target output	Error	Weight Adjustments			Weights		
	x_0	x_1	x_2	y_{in}	t	$t - y_{in}$	Δw_0	Δw_1	Δw_2	w_0	w_1	w_2
0										.3	.3	.3
1	1	1	1	.9	1	.1	.02	.02	.02	.32	.32	.32
2	1	1	-1	.32	-1	-1.32	-.264	-.264	.264	.056	.056	.584
3	1	-1	1	.584	-1	-1.584	-.317	.317	-.317	-.261	.373	.267
4	1	-1	-1	-.901	-1	-.099	-.02	.02	.02	-.281	.393	.287

Table 6.12. *Performance of the net after LMS learning*

#	Input Pattern			Net input	Output	Target Output
	x_0	x_1	x_2	$y_{in} = \sum w_i x_i$	y_{out}	t
1	1	1	1	.399 > 0	1	1
2	1	1	-1	-.175 < 0	-1	-1
3	1	-1	1	-.387 < 0	-1	-1
4	1	-1	-1	-.961 < 0	-1	-1

(d) Extended Delta Rule

The Extended Delta Rule removes the restriction of the output activation function being the identity function only. Any differentiable function can be used for this purpose. Here the weight adjustment is given by

$$\Delta w_{ij} = \eta \times (t_j - y_{out_j}) \times x_i \times g'(y_{in_i}) \quad (6.50)$$

where $g(\cdot)$ is the output activation function and $g'(\cdot)$ is its first derivative.

6.7.2 Unsupervised Learning

So far we have considered only supervised learning where the training patterns are provided with the target outputs. However, the target output may not be available during the learning phase.

Typically, such a situation is posed by a clustering problem, rather than a classification or association problem.

Let us, for instance, consider a set of points on a Cartesian plane as shown in Fig. 6.38 (a). The problem is to divide the given patterns into two clusters so that when the neural net is presented with one of these patterns, its output indicates the cluster to which the pattern belongs. Intuitively, the patterns those are *close* to each other should form a cluster. Of course we must have a suitable measure of *closeness*. Fig. 6.38(b) shows the situation after the neural net learns to form the clusters, so that it ‘knows’ the cluster to which each of the given pattern belongs.

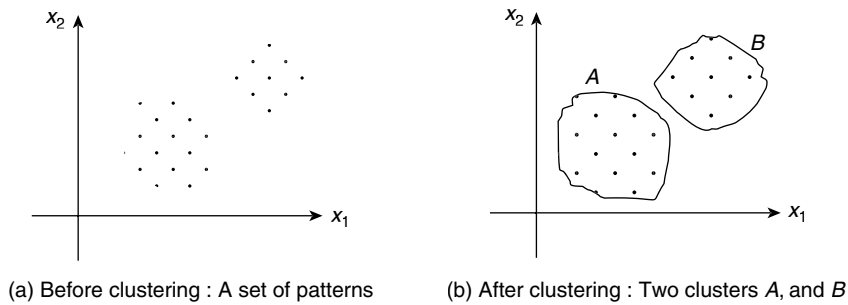


Fig. 6.38. Clustering a given set of patterns

Clustering is an instance of unsupervised learning because it is not assisted by any *teacher*, or, any target output. The network itself has to *understand* the patterns and put them into appropriate clusters. The only clue is the given number of clusters. Let us suppose that the network output layer has one unit for each cluster. In response to a given input pattern, exactly one among the output units has to fire. To ensure this, additional features need to be included in the network so that the network is compelled to make a decision as to which unit should fire due to certain input pattern. This is achieved through a mechanism called competition. The most commonly used competitive learning mechanism is the so called winner-takes-all learning. The basic principle of winner-take-it-all is discussed below.

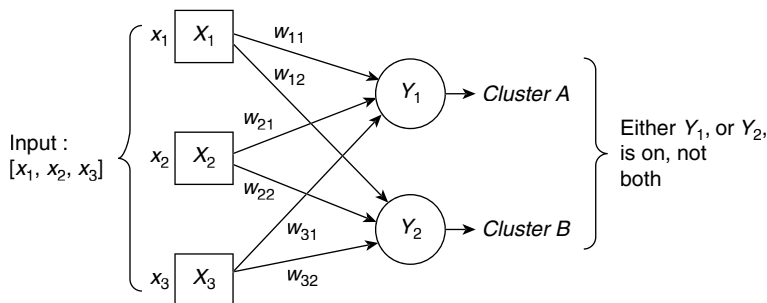


Fig. 6.39. A 3 input 2 output clustering network

Winner-takes-all Let us consider a simple ANN consisting of three input units and two output units as shown in Fig. 6.39. As each input unit of a clustering network corresponds to a component of the input vector, this network accepts input vectors of the form $[x_1, x_2, x_3]$. Each output unit of a clustering network represents a cluster. Therefore the present network will divide the set of input patterns into two clusters.

The weight vector for an output unit in a clustering network is known as the exemplar or code-book vector for the vectors of the cluster represented by the corresponding output unit. Hence, for the net in Fig. 6.39, the weight vector $[w_{11}, w_{21}, w_{31}]$ is an exemplar (or code-book vector) for the patterns of cluster A. Similarly, $[w_{12}, w_{22}, w_{32}]$ is the exemplar for the patterns of cluster B.

In the winner-takes-all strategy, the network finds the output unit that matches best for the current input vector and makes it the winner. The weight vector for the winner is then updated according to the learning algorithm. One way of deciding the winner is to employ the square of the Euclidean distance between the input vector and the exemplar. The unit that has the smallest Euclidean distance between its weight vector and the input vector is chosen as the winner. The algorithmic steps for an m -input n -output clustering network are given below :

1. For each output unit $Y_j, j = 1$ to n , compute the squared Euclidean distance as

$$D(j) = \sum_{i=1}^m (w_{ij} - x_i)^2.$$

2. Let Y_j be the output unit with the smallest squared Euclidean distance $D(j)$.
3. For all output units within a specified neighbourhood of Y_j , update the weights as follows :

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \eta \times [x_i - w_{ij}(\text{old})]$$

The concepts of neighbourhood, and winner-takes-all, in general, will be discussed in greater detail in the chapter on competitive neural nets. Example 6.9 illustrates the winner-takes-all strategy more explicitly..

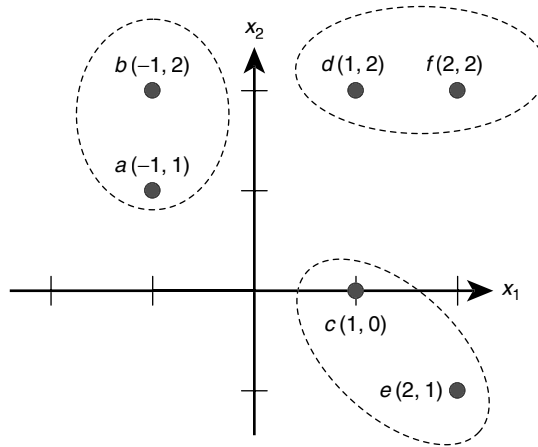


Fig. 6.40. A clustering problem showing the expected clusters

Example 6.9 (Competitive learning through Winner-takes-all strategy)

Let us consider a set of six patterns $S = \{a(-1, 1), b(-1, 2), c(1, 0), d(1, 2), e(2, -1), f(2, 2)\}$. The positions of these points on a two dimensional Euclidian plane are shown in Fig. 6.40. Intuitively, the six points form three clusters $\{a, b\}$, $\{d, f\}$, and $\{c, e\}$. Given the number of clusters to be formed as 3, how should a neural net learn the clusters by applying the winner-takes-all as its learning strategy? Let us see.

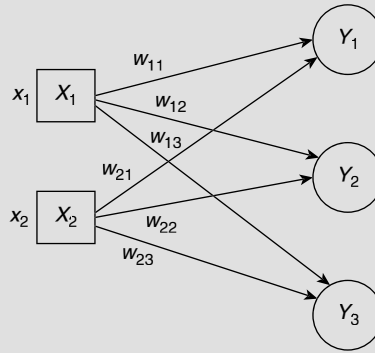


Fig. 6.41. A 2 input 3 output ANN to solve the clustering problem of Example 6.9

The input patterns are of the form (x_1, x_2) and the whole data set is to be partitioned into three clusters. So the target ANN should have two input units and three output units (Fig. 6.41). Let the initial distribution of (randomly chosen) weights be

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} = \begin{bmatrix} 0 & -1 & 2 \\ 2 & 0 & 1 \end{bmatrix}$$

This means that the exemplars, or code vectors, are initially $C_1 = (w_{11}, w_{21}) = (0, 2)$, $C_2 = (w_{12}, w_{22}) = (-1, 0)$, and $C_3 = (w_{13}, w_{23}) = (2, 1)$. The positions of the code vectors as well as the patterns to be clustered are shown in Fig. 6.42.

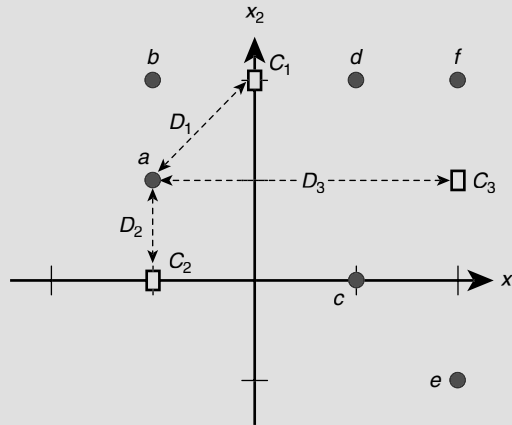


Fig. 6.42. Initial positions of the code vectors

Clusters are formed on the basis of the distances between a pattern and a code vector. For example, to determine the cluster for the pattern a $(-1, 1)$ we need to compute the Euclidean distance between the point a and each code vector C_1, C_2, C_3 . The pattern is then clustered with nearest among the three code vectors. Let D_1, D_2, D_3 be the squares of the Euclidean distances between a pattern and C_1, C_2, C_3 respectively. For a $(-1, 1)$ the computations of D_1, D_2, D_3 are as follows.

$$D_1 = (w_{11} - x_1)^2 + (w_{21} - x_2)^2 = (0 + 1)^2 + (2 - 1)^2 = 2$$

$$D_2 = (w_{12} - x_1)^2 + (w_{22} - x_2)^2 = (-1 + 1)^2 + (0 - 1)^2 = 1$$

$$D_3 = (w_{13} - x_1)^2 + (w_{23} - x_2)^2 = (2 + 1)^2 + (1 - 1)^2 = 9$$

Since D_2 is the minimum among these the pattern a $(-1, 1)$ the corresponding exemplar C_2 is declared the winner and the pattern is clustered around the code vector C_2 . Similar computations for the pattern b $(-1, 2)$ are given below.

$$D_1 = (w_{11} - x_1)^2 + (w_{21} - x_2)^2 = (0 + 1)^2 + (2 - 2)^2 = 1$$

$$D_2 = (w_{12} - x_1)^2 + (w_{22} - x_2)^2 = (-1 + 1)^2 + (0 - 2)^2 = 4$$

$$D_3 = (w_{13} - x_1)^2 + (w_{23} - x_2)^2 = (2 + 1)^2 + (1 - 2)^2 = 10$$

In this case D_1 is the minimum making C_1 the winner. Table 6.13 summarizes the initial clustering process yielding the clusters $C_1 = \{b, d\}$, $C_2 = \{a\}$ and $C_3 = \{c, e, f\}$ (Fig. 6.43). The minimum distance is indicated by a pair of parenthesis.

Table 6.13. Initial cluster formation

#	Pattern	Squared Euclidean distance			Minimum of D_1, D_2, D_3	Winner / Cluster
		D_1	D_2	D_3		
1	a $(-1, 1)$	2	(1)	9	D_2	C_2
2	b $(-1, 2)$	(1)	4	10	D_1	C_1
3	c $(1, 0)$	5	4	(2)	D_3	C_3
4	d $(1, 2)$	(1)	8	2	D_1	C_1
5	e $(2, -1)$	13	10	(4)	D_3	C_3
6	f $(2, 2)$	4	13	(1)	D_3	C_3

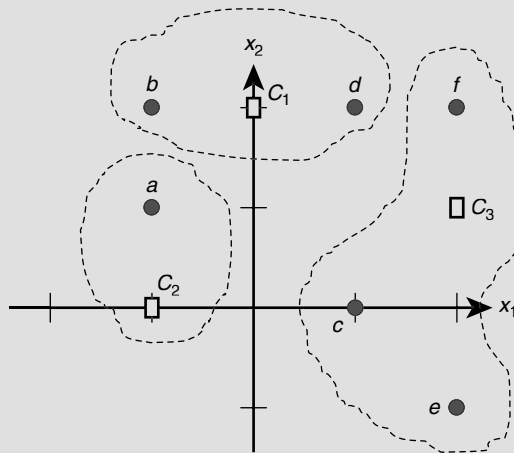


Fig. 6.43. Initial clusters

The initial clustering depicted in Fig. 6.43 is neither perfect, nor final. This will change as the learning proceeds through a number of epochs. This is explained below.

We take the value of the learning rate $\eta = 0.5$ and follow the steps of winner-takes-all strategy to modify the positions of the exemplars so that they represent the respective clusters better. The process is described below.

(a) 1st Epoch

- (i) Find the winner for the pattern $a(-1, 1)$ and adjust the corresponding code-vector

$$D_1 = (w_{11} - x_1)^2 + (w_{21} - x_2)^2 = (0 + 1)^2 + (2 - 1)^2 = 2$$

$$D_2 = (w_{12} - x_1)^2 + (w_{22} - x_2)^2 = (-1 + 1)^2 + (0 - 1)^2 = 1$$

$$D_3 = (w_{13} - x_1)^2 + (w_{23} - x_2)^2 = (2 + 1)^2 + (1 - 1)^2 = 9$$

Since D_2 is the minimum among these, the winner is C_2 . Hence the weights w_{12} and w_{22} are to be changed. The new weights are obtained as

$$w_{12}(\text{new}) = w_{12}(\text{old}) + \eta(x_1 - w_{12}(\text{old})) = -1 + .5 \times (-1 + 1) = -1$$

$$w_{22}(\text{new}) = w_{22}(\text{old}) + \eta(x_2 - w_{22}(\text{old})) = 0 + .5 \times (1 - 0) = .5$$

The new code vector C_2 is $(-1, .5)$. It is closer to the training vector $a(-1, 1)$ than the old code vector $(-1, 0)$ (Fig. 6.44(b)). The weight matrix now changes to

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} = \begin{bmatrix} 0 & -1 & 2 \\ 2 & .5 & 1 \end{bmatrix}$$

- (ii) Find the winner for the pattern $b(-1, 2)$ and adjust the corresponding code-vector

$$D_1 = (w_{11} - x_1)^2 + (w_{21} - x_2)^2 = (0 + 1)^2 + (2 - 2)^2 = 1$$

$$D_2 = (w_{12} - x_1)^2 + (w_{22} - x_2)^2 = (-1 + 1)^2 + (.5 - 2)^2 = 2.25$$

$$D_3 = (w_{13} - x_1)^2 + (w_{23} - x_2)^2 = (2 + 1)^2 + (1 - 2)^2 = 10$$

Since D_1 is the minimum among these, the winner is C_1 . Hence the weights w_{11} and w_{21} are to be changed. The new weights are obtained as

$$w_{11}(\text{new}) = w_{11}(\text{old}) + \eta(x_1 - w_{11}(\text{old})) = 0 + .5 \times (-1 - 0) = -.5$$

$$w_{21}(\text{new}) = w_{21}(\text{old}) + \eta(x_2 - w_{21}(\text{old})) = 2 + .5 \times (2 - 2) = 2$$

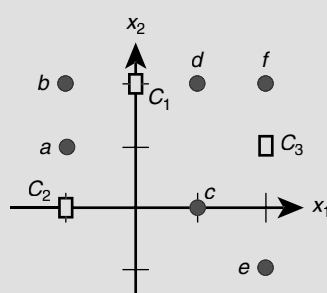
The new code vector C_1 is $(-.5, 2)$. It is closer to the training vector $b(-1, 2)$ than the old code vector $(0, 2)$ (Fig. 6.44(c)). The new weight matrix is

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} = \begin{bmatrix} -.5 & -1 & 2 \\ 2 & .5 & 1 \end{bmatrix}$$

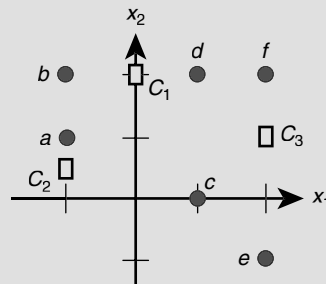
The successive iterations of the 1st epoch with the rest of the training vector are carried out in a similar fashion and are summarized in Table 6.14. The corresponding changes in the positions of the code vectors are depicted in Fig. 6.44(b)-(g).

Table 6.14. Clustering process during the 1st epoch

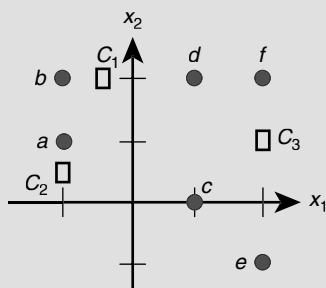
#	Training Patterns	Squared Euclidean Distance			Winner	New Code Vectors		
		D_1	D_2	D_3		C_1 (w_{11}, w_{21})	C_2 (w_{12}, w_{22})	C_3 (w_{13}, w_{23})
0						(0, 2)	(-1, 0)	(2, 1)
1	a (-1, 1)	2	(1)	9	C_2	(0, 2)	(-1, .5)	(2, 1)
2	b (-1, 2)	(1)	2.25	10	C_1	(-.5, 2)	(-1, .5)	(2, 1)
3	c (1, 0)	6.25	4.25	(2)	C_3	(-.5, 2)	(-1, .5)	(1.5, .5)
4	d (1, 2)	(2.25)	6.25	2.5	C_1	(.25, 2)	(-1, .5)	(1.5, .5)
5	e (2, -1)	12.06	11.25	(2.5)	C_3	(.25, 2)	(-1, .5)	(1.75, -.25)
6	f (2, 2)	(3.06)	11.25	5.13	C_1	(1.13, 2)	(-1, .5)	(1.75, -.25)



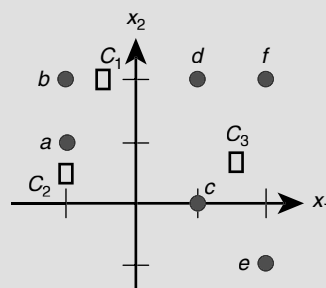
(a) Initial positions



(b) Iteration #1: Pattern a, C_2 relocated



(c) Iteration #2: Pattern b, C_1 relocated



(d) Iteration #3: Pattern c, C_3 relocated

Fig. 6.44. Successive positions of the code vectors during the 1st epoch of the clustering process

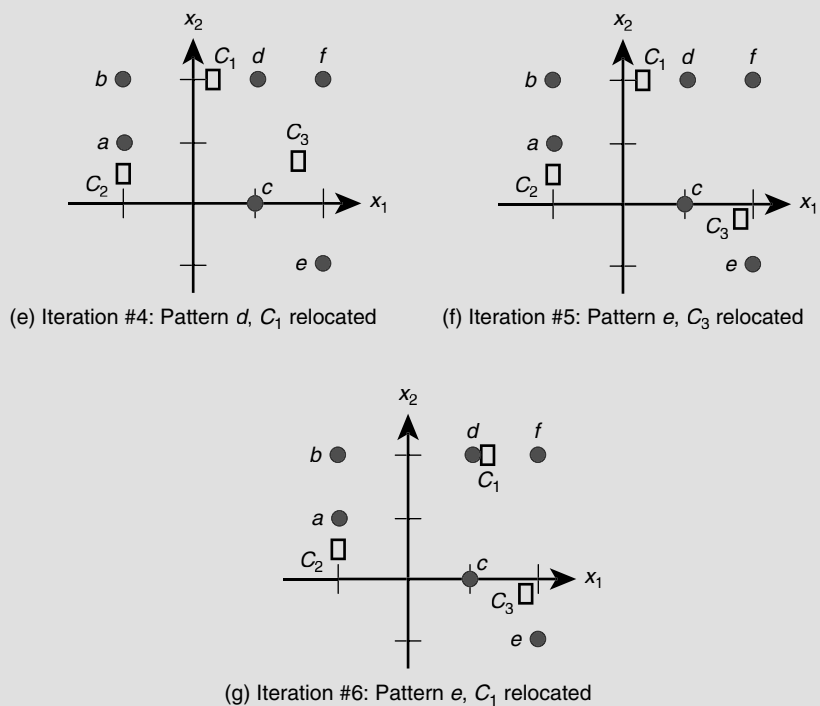


Fig. 6.44. (Continued)

Table 6.15. Clusters after the 1st epoch

#	Training Vector	Cluster
1	$a(-1, 1)$	C_2
2	$b(-1, 2)$	C_2
3	$c(1, 0)$	C_3
4	$d(1, 2)$	C_1
5	$e(2, -1)$	C_3
6	$f(2, 2)$	C_1

Clusters formed

 C_1 : $\{d, f\}$ C_2 : $\{a, b\}$ C_3 : $\{c, e\}$

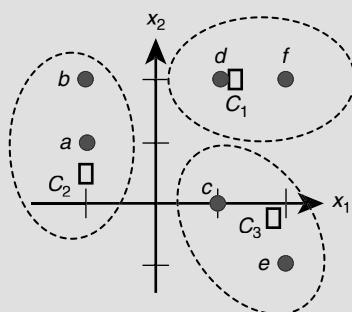


Fig. 6.45. Clusters formed after the 1st epoch

Therefore, the weight matrix at the end of the first epoch is obtained as

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix} = \begin{bmatrix} 1.13 & -1 & 1.75 \\ 2 & .5 & -.25 \end{bmatrix}$$

The corresponding clusters can be easily found by computing the distance of each training vector from the three code vectors and attaching it to the cluster represented by the nearest code vector. Table 6.15 summarizes these results and Fig. 6.45 shows the clusters formed at the end of the 1st epoch. It is obvious from Fig. 6.45 that the clusters formed conform to our intuitive expectation. However, this should be further consolidated by a second epoch of training.

Table 6.16. Clustering process during the 2nd epoch

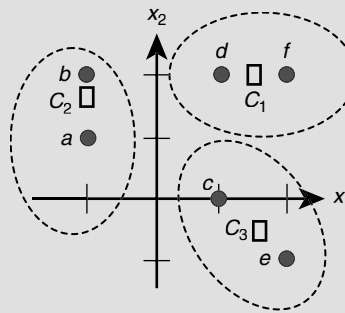
#	Training Patterns	Squared Euclidean Distance			Winner	New Code Vectors		
		D_1	D_2	D_3		C_1 (w_{11}, w_{21})	C_2 (w_{12}, w_{22})	C_3 (w_{13}, w_{23})
0						(1.13, 2)	(-1, .5)	(1.75, -.25)
1	a (-1, 1)	5.54	(.25)	9.13	C_2	(1.13, 2)	(-1, 1.5)	(1.75, -.25)
2	b (-1, 2)	4.54	(.25)	12.63	C_2	(1.13, 2)	(-1, 1.75)	(1.75, -.25)
3	c (1, 0)	4.02	7.06	(.63)	C_3	(1.13, 2)	(-1, 1.75)	(1.38, -.13)
4	d (1, 2)	(.02)	4.06	4.68	C_1	(1.07, 2)	(-1, 1.75)	(1.38, -.13)
5	e (2, -1)	9.86	16.63	(1.14)	C_3	(1.07, 2)	(-1, 1.75)	(1.69, -.57)
6	f (2, 2)	(.86)	9.06	6.7	C_1	(1.56, 2)	(-1, 1.75)	(1.69, -.57)

The summary of the second epoch of training is given in Table 6.16. Table 6.17 shows the clusters formed at the end of this epoch and Fig. 6.46 depicts the location of the code vectors as well as the corresponding clusters. It should be noted that though the clusters has not changed from the first epoch, the positions of the exemplars has changed. In fact, these have moved towards the 'centre' of the respective clusters. This is expected, because the exemplar is the representative of a cluster.

Table 6.17. *Clusters after the 2nd epoch*

#	Training Vector	Cluster
1	a (-1, 1)	C_2
2	b (-1, 2)	C_2
3	c (1, 0)	C_3
4	d (1, 2)	C_1
5	e (2, -1)	C_3
6	f (2, 2)	C_1

Clusters formed

 C_1 : {d, f} C_2 : {a, b} C_3 : {c, e}**Fig. 6.46.** *Clusters formed after the 2nd epoch*

CHAPTER SUMMARY

- ANNs are inspired by the biological brain that processes data in terms of patterns rather than sequential fetch and execute cycle of instruction execution by a classical Von Neuman digital computer.
- Information is stored in brain as strengths of the synaptic gaps between the neurons. Similarly, knowledge is stored in an ANN as weights of interconnection between neural processing units. Thus both the brain and the ANN stores information in a distributed manner.
- ANNs are suitable for problem related to pattern classification and pattern association.
- The earliest artificial neural model was proposed by McCulloch & Pitts in 1943.
- The perceptron model was proposed by Rosenblatt in 1962. It has the nice property of being able to learn to classify a linearly separable set of patterns.
- ANNs have various architectures, such as, single-layer feed forward, multi-layer feed forward, competitive networks, recurrent networks etc.
- Certain functions such as the identity function, the step function, the sigmoid function, the hyperbolic tangent function etc. are widely used as activation function in an ANN.

- There are two kinds of learning methods for an ANN, supervised and unsupervised. Learning takes place with the help of a set of training patterns. Under supervised learning each training pattern is accompanied by its target output. These target outputs are not available during unsupervised learning.
- The formulae for interconnection weight adjustment Δw_i for different learning rules are shown in Table 6.18.

Table 6.18. Summary of ANN learning rules

#	Learning Rule	Formula for Δw_i
1	Hebb	$\Delta w_i = x_i \times t$
2	Perceptron	$\Delta w_i = \eta \times (t - y_{out}) \times x_i$
3	Delta/LMS/Widrow-Hoff	$\Delta w_i = \eta \times (t - y_{in}) \times x_i$
4	Extended delta	$\Delta w_{ij} = \eta \times (t_j - y_{out_j}) \times x_i \times g'(y_{in_j})$
5	Winner-takes-all	$\Delta w_{ij} = \eta \times [x_i - w_{ij}(old)]$

- The weight vector associated with an output of a clustering ANN is called the exemplar or the code-book vector. It represents the cluster corresponding to the output unit of the clustering ANN.

? SOLVED PROBLEMS

Problem 6.1 Write MATLAB Code to realize the logical AND function with a neural net that learns the desired function through Hebb learning.

Solution 6.1 The MATLAB code and the output are given below as Fig. 6.47 and Fig. 6.48.

```
%MATLAB Code to realize the logical AND function with a neural net that
%learns the desired function through Hebb learning.
clear;
clc;
Inp1=[1 1 1];
Inp2=[1 1 -1];
Inp3=[1 -1 1];
Inp4=[1 -1 -1];
Mat(1,1:3)=Inp1;
Mat(2,1:3)=Inp2;
Mat(3,1:3)=Inp3;
Mat(4,1:3)=Inp4;
wt(1:3)=0;
Tar_Act=[1 -1 -1 -1];
bias=1;
for i=1:4
    wt=wt+Mat(i,1:3)*Tar_Act(i);
    bias=bias+Tar_Act(i);
    disp('Weight Matrix');
    disp(wt);
end
```

```
disp('Bias');
disp(bias);
end
disp('*****Final Weight Matrix*****');
disp(wt);
disp('*****Bias*****');
disp(bias);
```

Fig.6.47. MATLAB code for Hebb learning of AND function by an ANN

Results

```

Weight Matrix
    1    1    1
Bias
    2
Weight Matrix
    0    0    2
Bias
    1
Weight Matrix
   -1    1    1
Bias
    0
Weight Matrix
   -2    2    2
Bias
   -1
*****Final Weight Matrix*****
   -2    2    2
*****Bias*****
   -1

```

Fig. 6.48. MATLAB results for Hebb learning of AND function

Problem 6.2 Write MATLAB Code to realize the logical AND function with a neural net that learns the desired function through Perceptron learning.

Solution 6.2 The MATLAB code for the purpose is given below in Fig. 6.49. Fig. 6.50 presents the corresponding screenshot.

```
% Example 1.7: Design a perceptron and train it to realize the logical AND
function.
P = [0 0 1 1; 0 1 0 1];           % Possible values of 2 variables in a ma-
                                   % trix format
T = [0 0 0 1];                     % Expected outputs for above dataset
```

```

net = newp([0 1; 0 1],1);           % Creates network with two inputs and 1
                                   % output with ranges of values
net.trainParam.epochs = 20;         % Sets the number of maximum iterations
net = train(net,P,T);               % Trains the network
simulation = sim(net,P)              % Simulates neural network
plotpv(P,T)                         % Plot input/target vectors
plotpc(net.iw{1,1},net.b{1})        % Plot classification line

```

Fig. 6.49. MATLAB code for perceptron learning of AND function

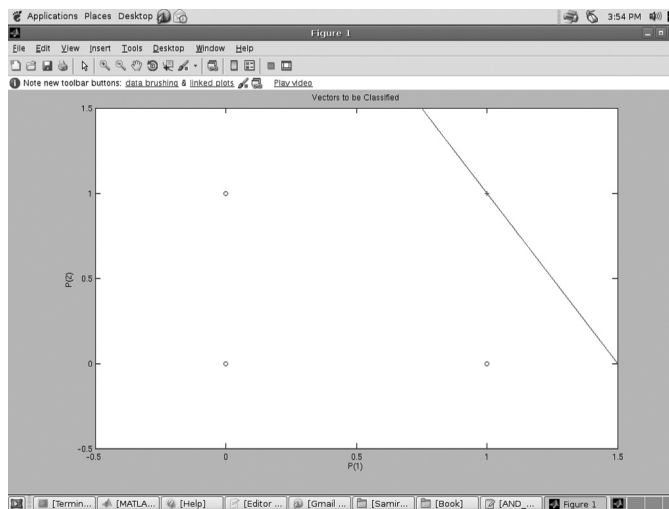


Fig. 6.50. MATLAB output for perceptron implementing AND function

Problem 6.3 Write MATLAB Code to realize the logical AND function with a neural net that learns the desired function through LMS learning.

Solution 6.3 The MATLAB code for the purpose is given in Fig. 6.51. Fig. 6.52 shows the results and screenshot is shown in Fig. 6.53.

```

%Matlab Code to implement LMS learning
clear;
clc;
Inp1=[1 1 1];
Inp2=[1 1 -1];
Inp3=[1 -1 1];
Inp4=[1 -1 -1];
Mat(1,1:3)=Inp1;
Mat(2,1:3)=Inp2;
Mat(3,1:3)=Inp3;
Mat(4,1:3)=Inp4;

```

```

wt(1:3)=0.3;
wt_new(1:3)=0;
y_in(1:4)=0;
Tar_Act=[1 -1 -1 -1];
bias=1;
Learn_Rate=0.2;
for j=1:4
y_in(j)=wt(1)*Mat(j,1)+wt(2)*Mat(j,2)+wt(3)*Mat(j,3);
    for i=1:3
        wt_new(i)=wt(i)+ (Learn_Rate*(Tar_Act(j)-y_in(j))*Mat(j,i));
        wt(i)=wt_new(i);
        disp('Weight Matrix');
        disp(wt);
    end
end
disp('*****Final Weight Matrix*****');
disp(wt);

```

Fig. 6.51. MATLAB code for LMS learning of AND function by an ANN

```

Weight Matrix
    0.3200    0.3000    0.3000
Weight Matrix
    0.3200    0.3200    0.3000
Weight Matrix
    0.3200    0.3200    0.3200
Weight Matrix
    0.0560    0.3200    0.3200
Weight Matrix
    0.0560    0.0560    0.3200
Weight Matrix
    0.0560    0.0560    0.5840
Weight Matrix
   -0.2608    0.0560    0.5840
Weight Matrix
   -0.2608    0.3728    0.5840
Weight Matrix
   -0.2608    0.3728    0.2672
Weight Matrix
   -0.2806    0.3728    0.2672
Weight Matrix
   -0.2806    0.3926    0.2672
Weight Matrix
   -0.2806    0.3926    0.2870
*****Final Weight Matrix*****
   -0.2806    0.3926    0.2870

```

Fig. 6.52. MATLAB results for LMS learning of AND function

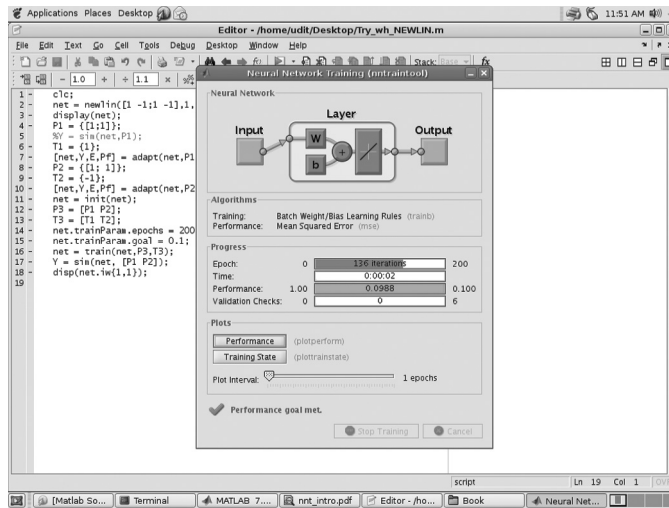


Fig. 6.53. MATLAB screen showing results of LMS learning of AND function

Problem 6.4 Write MATLAB Code to implement the Winner-takes-all strategy of pattern clustering to cluster the .

Solution 6.4 The MATLAB code shown in Fig. 6.54.

```

clear;
clc;
lr = 0.5;

Inp = [1 0 -1;1 -1 0];
wt = [0 1 2;-1 1 2];
[rowInp, colInp] = size(Inp);
[rowwt, colwt] = size(wt);
Res = 0;
High_Indx = 0;
for i = 1:colInp
    Inp(:,i) = Inp(:,i)/norm(Inp(:,i));
    for j = 1:colwt
        wt(:,j) = wt(:,j)/norm(wt(:,j));
        matrix_mult = dot(wt(:,j),Inp(:,i));
        if matrix_mult > Res
            Res = matrix_mult;
            High_Indx = j;
            disp (wt(:,j))
        end
    end
end

wt(:,High_Indx) = wt(:,High_Indx) + lr*(Inp(:,i) - wt(:,High_Indx));
disp('*****Weights*****');
wt(:,High_Indx) = wt(:,High_Indx)/norm(wt(:,High_Indx));
end

```

```
figure
plot(Inp(1,:), Inp(2,:), wt(1,:), wt(2,:))
axis([-max(max(abs(Inp)))-0.2max(max(abs(Inp)))+0.2-
max(max(abs(Inp)))-0.2 max(max(abs(Inp))+0.2])
```

Fig. 6.54. MATLAB code for clustering through winner-takes-all

✓ TEST YOUR KNOWLEDGE

- 6.1 Which of the following parts of a biological neuron is modeled by the weighted interconnections between the input units and the output unit of an artificial neural model?
 - a) Dendrite
 - b) Axon
 - c) Soma
 - d) Synapse
- 6.2 The effect of the synaptic gap in a biological neuron is modeled in artificial neuron model as
 - a) The weights of the interconnections
 - b) The activation function
 - c) The net input to the processing element
 - d) None of the above
- 6.3 In an artificial neural model, the activation function of the input unit is
 - a) The step function
 - b) The identity function
 - c) The sigmoid function
 - d) None of the above
- 6.4 Which of the following is not true about Perceptrons ?
 - a) It can classify linearly separable patterns
 - b) It does not have any hidden layer
 - c) It has only one output unit
 - d) None of the above
- 6.5 Recognition of hand written characters is an act of
 - a) Pattern classification
 - b) Pattern association
 - c) Both (a) & (b)
 - d) None the above
- 6.6 Identification of an object, e.g., a chair, a tree, or a human being, from the visual image of our surroundings, is an act of
 - a) Pattern classification
 - b) Pattern association
 - c) Both (a) & (b)
 - d) None the above
- 6.7 The interconnections of a Hopfield network are
 - a) Unidirectional
 - b) Bidirectional
 - c) Both (a) & (b)
 - d) None the above
- 6.8 The interconnections of a perception are
 - a) Unidirectional
 - b) Bidirectional
 - c) Both (a) & (b)
 - d) None the above
- 6.9 Parallel relaxation is a process related to the functionality of
 - a) Perceptrons
 - b) McCulloch-Pitts neurons
 - c) Hopfield networks
 - d) None the above
- 6.10 In which of the following ANN models the inhibition is absolute, i.e., a single inhibitive input can prevent the output to fire, irrespective of the number of excitatory inputs?

- a) Perceptrons
 - b) Hopfield network
 - c) McCulloch-Pitts neurons
 - d) None the above
- 6.11 Which of the following is not true about McCulloch-Pitts neurons ?
- a) The interconnections are unidirectional
 - b) All excitatory interconnections have the same weight
 - c) All inhibitory interconnections have the same weight
 - d) The activation is bipolar
- 6.12 Which of the following operations can be realized by a network of McCulloch-Pitts neurons, but not a network of perceptions?
- a) Logical AND
 - b) Logical OR
 - c) Logical XOR
 - d) None the above
- 6.13 Which of the following kinds of classification problems can be solved by a perception?
- a) Linearly separable
 - b) Non-linearly separable
 - c) Both (a) and (b)
 - d) None the above
- 6.14 Which of the following entities is guaranteed by the Perceptron Convergence Theorem to converge during the learning process ?
- a) The output activation
 - b) The interconnection weights
 - c) Both (a) and (b)
 - d) None the above
- 6.15 The XOR function can not be realized by a Perceptron because the input patterns are
- a) Not bipolar
 - b) Not linearly separable
 - c) Discrete
 - d) None the above
- 6.16 The XOR function can be realized by
- a) A Perceptron
 - b) A network of Perceptrons
 - c) A Hopfield network
 - d) None the above
- 6.17 Which of the following ANN architectures contains bidirectional interconnections?
- a) Single-layered feed forward
 - b) Multi-layered feed forward
 - c) Competitive networks
 - d) None the above
- 6.18 Which of the following activation functions is not differentiable ?
- a) Identity function
 - b) Heaviside function
 - c) Sigmoid function
 - d) None the above
- 6.19 Which of the following ANN learning algorithms is not a supervised learning ?
- a) Perceptron learning
 - b) Widrow-Hoff learning
 - c) Winner-takes-all
 - d) None the above
- 6.20 During learning, if a Perceptron misclassifies a training data negatively, i.e., erroneously yields an output -1 instead of $+1$, the interconnection weights are to be
- a) Increased
 - b) Decreased
 - c) Kept unaltered
 - d) None the above
- 6.21 During learning, if a Perceptron misclassifies a training data positively, i.e., erroneously yields an output $+1$ instead of -1 , the interconnection weights are
- a) Increased
 - b) Decreased
 - c) Kept unaltered
 - d) None the above
- 6.22 Which of the following learning rules does not guarantee to learn a classification instance even if the classes are linearly separable ?
- a) Perceptron learning rule
 - b) Hebb rule
 - c) Both (a) and (b)
 - d) None of the above

- 6.23 Which of the following is a competitive learning method ?
 a) Winner-takes-all b) Least-Mean-square
 c) Extended delta d) None the above
- 6.24 The weight vector attached to an output unit of a clustering network is known as
 a) an exemplar b) a code vector
 c) Both (a) and (b) d) None of the above
- 6.25 Which of the following ANN learning methods use Euclidean distance between the weight vector and the input vector to compute the output ?
 a) Perceptron learning b) Widrow-Hoff learning
 c) Winner-takes-all learning d) None of the above

Answers:

- | | | | | |
|----------|----------|----------|----------|----------|
| 6.1 (d) | 6.2 (a) | 6.3 (b) | 6.4 (d) | 6.5 (b) |
| 6.6 (a) | 6.7 (b) | 6.8 (a) | 6.9 (c) | 6.10 (c) |
| 6.11 (d) | 6.12 (d) | 6.13 (a) | 6.14 (b) | 6.15 (b) |
| 6.16 (b) | 6.17 (c) | 6.18 (b) | 6.19 (c) | 6.20 (a) |
| 6.21 (b) | 6.22 (b) | 6.23 (a) | 6.24 (c) | 6.25 (c) |

**EXERCISES**

- 6.1 There are two ways to interpret the role of the brain while considering the human body as a computational agent. One of them is to view the entire brain as a single unit that acts as the CPU for the whole body. The other is to consider the brain as a huge network of billions of processing units called neurons. Compare and contrast between these two views from a computational perspective.
- 6.2 The majority function outputs a 0 if there are more 0s in the inputs than 1s, and outputs a 1 if there are more 1s in the inputs than 0s. In short, it returns the majority input to the output. The truth table for a 3 input majority function is given as Table 6.19. Design a McCulloch-Pitts neural net to realize the 3-input majority function.

Table 6.19. *Three input majority function*

#	Inputs			Output $M(x_1, x_2, x_3)$
	x_1	x_2	x_3	
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

- 6.3 Consider the sets of points A and B shown in Fig. 6.47 where $A = \{a(-1, 1), b(-1, 0), c(-1, -1), e(0, 0), f(0, -1)\}$ and $B = \{d(0, 2), g(1, 2), h(1, 1), i(2, 2), j(2, 1)\}$. Propose two different Perceptrons to classify these sets of patterns by observation, analysis, and understanding only (and not by learning).

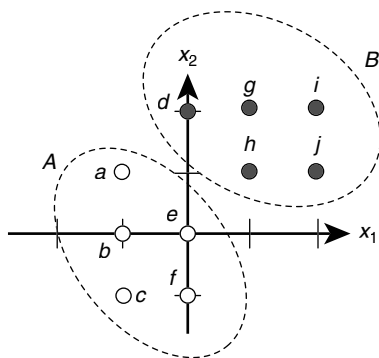


Fig. 6.47. A classification problem instance

- 6.4. Consider the 3-input majority function cited in Problem No. 2 of this Exercise. Fig. 6.48 shows the positions of the input patterns in a 3-dimensional space, classified on the basis of the corresponding output values. Are these patterns linearly separable? If so, propose a perceptron to realize this function.

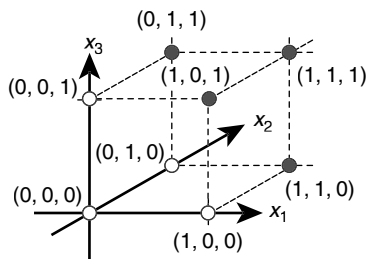


Fig. 6.48. Three input majority function

- 6.5 Draw the structure of a 3-3-2 multi-layered feed forward neural net. Also, present the matrix algebraic expression for the net inputs and the outputs of each layer.
- 6.6 Train a neural net through the Hebb rule to realize the logical OR function.
- 6.7 Realize the 3-input majority function through the Perceptron learning rule.
- 6.8 Train a neural net through the LMS rule to realize the logical OR function.
- 6.9 Let S be a set of six points on an Euclidean plane where $S = \{a(0, -1), b(0, -2), c(1, 2), d(1, 1), e(1, -1), f(2, 1)\}$ (Fig. 6.49). Taking the initial code vectors as $C_1 = (0, 0)$, and $C_2 = (-1, -2)$ find i) the initial clusters, and ii) final cluster with the help of Winner-takes-all strategy.

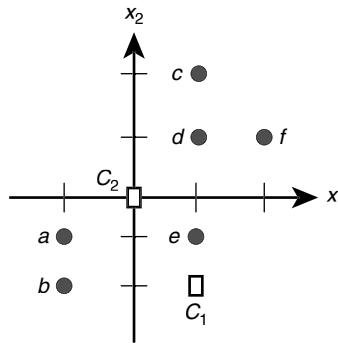


Fig. 6.49. A clustering problem with initial code vectors

BIBLIOGRAPHY AND HISTORICAL NOTES

Research in the area of ANNs started in early 40s, when neuro-physiologist, Warren McCulloch and mathematician Walter Pitts proposed their celebrated McCulloch-Pitts Theory of Formal Neural Networks. They even designed and developed a primitive ANN using simple electric circuits. The theories of McCulloch and Pitts on the nature and functionalities of neurons were further supported by Donald Hebb in his 1949 book 'The organization of behaviour'. In 1954 Marvin Minsky wrote his Ph.D thesis entitled 'Theory of Neural-Analog Reinforcement Systems and its Application to the Brain-Model Problem', which is widely acclaimed as an important early research into neural networks. Then in 1958 Frank Rosenblatt, a neuro-biologist at Cornell University, proposed the Perceptron, the first 'practical' ANN. However, Marvin Minsky and Seymour Papert proved in their book of 1969 entitled 'Perceptrons' that the capabilities of perceptrons are very limited. As a consequence, enthusiasm declined over the subsequent few years. The first neural networks that could be applied to real problems, ADALINE (ADaptive LINear Elements) and MADELINE (Multiple ADaptive LINear Elements), were developed by Bernard Wildrow and Marcian Hoff of Stanford University between 1959 and 1960. In 1982 John Hopfield of Caltech presented some new ideas in neural networks that eventually came to be known as Hopfield networks. A new era in neural networks started in 1986 when Rumelhart, Hinton and Williams proposed the back-propagation algorithm to overcome the limitations of perceptron. Many more developments took place in the last few decades in the field of ANNs. A few seminal works in this area are referred below.

Anderson, J. A., and Rosenfield, E., eds. (1988) 'Neurocomputing : Foundations of research'. Cambridge, MA, MIT Press.

Angeniol, B., Vaubois, G., and Le Texier, J-Y. (1988) 'Self-organizing feature maps and the traveling salesman problem'. Neural Networks, 1(4). pp. 289–293.

Block, H. D. (1962) 'The perceptron: A model for brain functioning.' Reviews for modern physics. 34, pp. 123–135.

Cohen, M. A. and Grossberg, S. (1983) 'Absolute stability of global pattern formation and parallel memory storage by competitive neural networks.' IEEE transactions on Systems, Man and Cybernetics, SMC-13, pp. 815–826.

- Hopfield, J. J. (1982) 'Neural networks and physical systems with emergent collective computational properties'. *Proc. of the National Academy of Sciences of the USA*, 79, pp. 2554–2588.
- Hopfield, J. J. and Tank, D. W. (1985) 'Neural computation of decisions in optimization Problems'. *Biological cybernetics*, 52(3), pp. 141–152.
- Hopfield, J.J. and Tank, D.W. (1986) 'Computing with neural circuits'. *Science*, 233, pp. 625–633.
- Grossberg, S. (1973) 'Contour enhancement, short-term memory, and constancies in reverberating neural networks'. *Studies in Applied Mathematics*, 52.
- Grossberg, S. (1982) 'Studies of mind and brain'. Boston, Reidel.
- Hebb, D. O. (1949) 'The organization of behaviour'. New York, John Wiley & sons.
- Kohonen, T. (1982) 'Self-organized formation of topologically correct feature maps'. *Biological cybernetics*, 43, pp. 59–69.
- McCulloch, W. and Pitts, W. (1943) 'A logical calculus of the ideas immanent in nervous activity'. *Bulletin of Mathematical Biophysics*, 7, pp.115–133.
- Minsky, M. and Papert, S. (1969) 'Perceptrons'. MIT Press.
- Rosenblatt, F. (1959) 'Two theorems of statistical separability in the perceptron'. *Mechanization of thought process : proc. of symposium at National Physical Laboratory, London*, pp. 421–456.
- Rosenblatt, F. (1962) 'Principles of Neurodynamics'. Spartan Books.
- Rumelhart, D. E., Hinton, G. E. and Williams, R. J. (1986) 'Learning representations by back-propagating errors'. *Nature*, 323, pp. 533–536.
- Tank, D. W., and Hopfield, J. J. (1987) 'Collective computation in neuronlike circuits'. *Scientific American*, 257, pp. 104–114.
- Von Neumann, J. (1958) 'The computer and the brain'. New Haven : Yale University Press.
- Widrow, B. and Hoff, M. E. (1960) 'Adaptive switching circuits'. *IRE WESCON convention record*, part 4, pp. 96–104.